

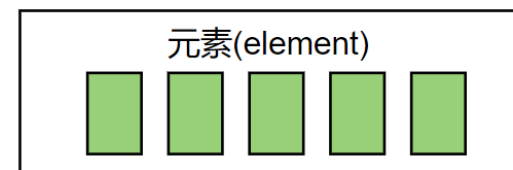
有序对象

主要内容

- 有序对象（字符串、range对象、列表和元组）的通用方法（第4章, 5.1, 5.2）
- 可变有序对象（列表）（第4章）
- 有序对象的切片（第6章）
- 序列解包(2.4)
- 用于序列的常用内置函数（4.8）：zip和enumerate
- 函数式编程：
 - 列表推导式（6.5-6.7）
 - 生成器表达式
 - 高阶函数（12.6, 12.7）

列表和元组

- 容器对象：可以容纳多个对象的对象，其每个组成部分称为**元素(element)**
- 有序对象(sequence)：容器对象的元素按照一定顺序排列，可访问指定位置（下标和切片）的元素
- 列表(list)是Python中内置**可变的有序序列**，元组(tuple)是python中内置**不可变的有序序列**
- **列表字面量**定义：开中括号表示列表的开始，配对的闭中括号表示列表的结束，列表的元素之间以逗号隔开，比如[1, 2, 3]
- **元组字面量**定义：比如(1, 2, 3)
 - 与列表字面量类似，开圆括号和闭圆括号表示元组的开始和结束
 - 在不引起歧义时，圆括号也可省略
 - **注意**：如果创建只有一个元素的元组，需要在元素后面加上一个逗号“,”，比如(1,)
- 列表和元组中的元素：
 - 可以是任意类型的数据(对象)，比如整数、浮点数、字符串等基本类型，也可是列表、元素、字典、集合以及其他自定义类型
 - 每个元素的类型也可各不相同



列表和元组字面量

- 内置函数 **len(obj)** 返回容器对象obj中的元素的个数

```
>>> s = []
>>> s
[]
>>> len(s)
0
>>> [1, 2, 3]
[1, 2, 3]
>>> s = [[1, 2, 3], [4, 5, 6]]
>>> s
[[1, 2, 3], [4, 5, 6]]
>>> len(s)
2

>>> ['tony', ('python', 'math'), 18]
['tony', ('python', 'math'), 18]
```

```
>>> t = ()
>>> t
()
>>> len(t)
0
>>> t = (2, [1, 2, 3], [4, 5, 6])
>>> t
(2, [1, 2, 3], [4, 5, 6])
>>> len(t)
3

>>> 1, 2
(1, 2)

>>> x = 1, 2, (3, 4)
>>> x
(1, 2, (3, 4))
>>> x = (3)
>>> x
3
>>> x = 3, # or (3, )
>>> x
(3,)
```

有序序列：下标访问

有序序列（列表、元组、range对象、字符串等）

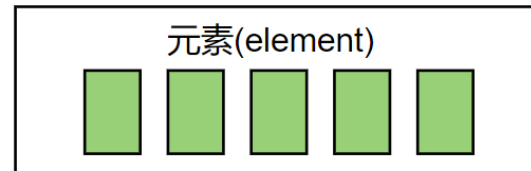
- 通过下标访问有序对象中对应位置的元素：`seq[index]`
- 合法下标为 $[-\text{len}(\text{seq}), \text{len}(\text{seq})-1]$ 。负数下标 $-i$ 等价于 $\text{len}(\text{seq}) - i$
- 如果下标越界，程序会报错(Exception `IndexError`)
- 字符串也是有序序列对象，也可通过下标访问，表示**第几个字符组成的字符串**

0	1	2	3	4	[0,5)
1	2	3	4	5	len(x)=5
-5	-4	-3	-2	-1	[-5,0)

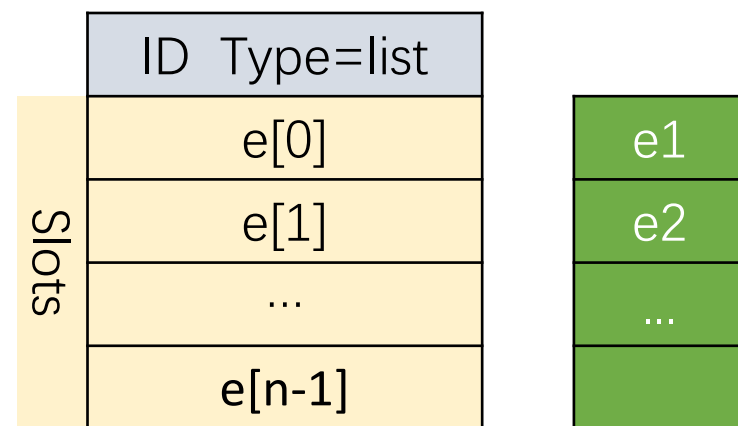
```
>>> x = [1, 2, 3, 4, 5]
>>> x[0]
1
>>> x[-1]
5
>>> x[len(x)]
...
IndexError: list index out of range
```

```
>>> t = (1, 2, 3, 4, 5)
>>> t[0]
1
>>> t[-5]
1
```

列表和元组



- 列表和元组作为容器对象，其元素可以是任何类型的对象
- 列表和元组的内部实现非常相似：
 - 列表和元组有多个slot，每个slot保存对应位置的对象(元素)的引用(相当于保存各个对象的ID)
 - 列表是可变对象，指的是可以改变列表对象的**各个slot**
 - 可以**增加和删除slot(元素)**
 - 增加：listobj.append(obj), listobj.extend(iterable), listobj.insert(idx, obj)
 - 删除：listobj.pop(idx) del listobj[idx]
 - 可以**(赋值) 修改slot(元素)**：listobj[0] = -1
 - 元组是不可变对象，指一旦创建，无法改变元组对象的**各个slot**
 - 无法**增加和删除slot(元素)**：没有相应的方法
 - 无法**(赋值) 修改slot(元素)**

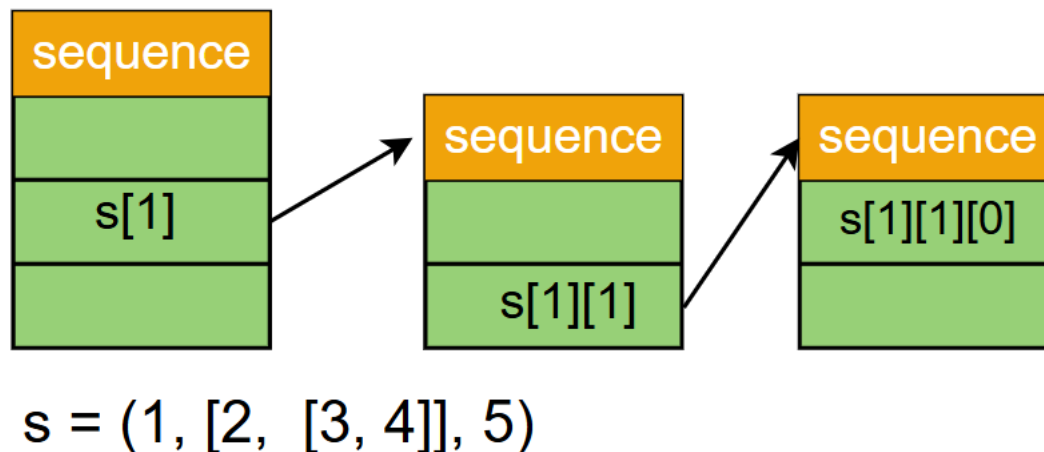


slot: id of element

多级下标

- **下标形式**`obj[expr]`表示某些可变容器对象(如列表,字典等)中的指定位置中的元素
- 列表和元组中的元素的类型没有限制。其元素可以是整数、浮点数等,也可是一个有序对象
 - 多级下标形式: `seq[index1][index2]`
 - 连续的下标访问采用**左结合**的方式
 - 通过`seq[index1]`得到`seq`下标为`index1`的元素, 比如为`e`
 - `seq[index1][index2]`相当于`e[index2]`, 即访问`seq` (下标为`index1`)元素所指向的容器对象中的相应(下标为`index2`)元素

```
>>> s = (1, [2, [3, 4]], 5)
>>> s[1]
[2, [3, 4]]
>>> s[1][1]
[3, 4]
>>> s[1][1][0]
3
```



赋值语句：从变量到属性到下标形式

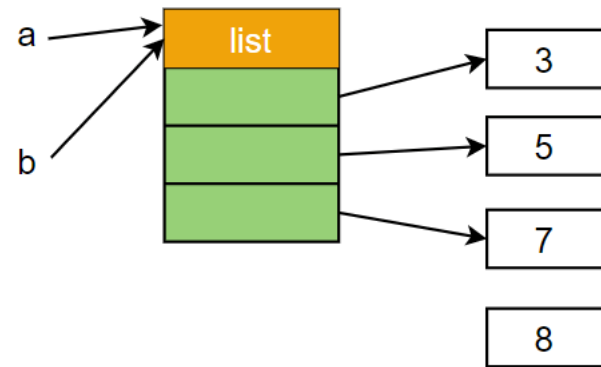
强调：赋值语句无法修改某个引用（名字或列表的 slot）所指向的对象的值，而是该引用指向右边表达式求解后的对象

- 赋值语句 LHS = RHS
 - LHS为变量var时,将名字空间中的名字与RHS给出的表达式运算后的对象绑定
 - 变量var出现在表达式, 表示该名字指向的对象
- 类似的, 属性形式obj.attr也可出现在LHS和表达式中, 比如import math; math.pi = 3.14
- **下标形式obj[expr]**表示某些可变容器对象(如列表,字典等)中的指定位置的元素, 也可出现在LHS和表达式
 - 其出现在表达式中, 表示访问该元素所指向的对象
 - 如果obj为列表, 则其出现在赋值语句的左边LHS时, 该列表对象的相应元素指向一个RHS求解后的对象
 - 为变量（保存对象的引用）赋值时, 并不是直接修改变量所指的对象的值, 而是使**变量指向新的值(对象)**
 - 为列表对象的元素赋值时, 并不是直接修改元素所指向的对象的值, 而是使**元素指向新的值(对象)**
 - 元组、字符串等都是不可变对象, 因此下标形式seq[idx]不允许出现在赋值语句的左边

```
>>> a = [3, 5, 7]
>>> b = a
>>> a[1]
5
>>> id(a[1])
1733441856
```

```
>>> a[1] = 8
>>> id(a[1])
1733441904
>>> a
[3, 8, 7]
>>> b
[3, 8, 7]
```

```
>>> t = (1, 2, 3)
>>> t[0]
1
>>> t[0] = -1
... TypeError: 'tuple'
object does not support
item assignment
```



有序对象：下标形式

- 列表为**可变对象**, 指其元素可变, 即可**(通过赋值语句)修改列表的元素, 可通过相应方法增加或删除列表的元素**
- 元组不可变, 指的是**其元素不可变** (不能增加元素, 不能删除元素, 不能通过赋值来改变元素, 即指向其他对象)
- 元组和列表的元素可指向可变或不可变对象
 - 如果列表或元组(s)的某个元素(如s[1])是一个可变的容器对象 (如列表)
 - 可通过两层下标或相应方法来修改该元素所指向对象的值: s[1][0] = 4; s[1].append(0)
 - 但这点与外层容器对象(x, 可以是列表或元组)的可变性无关
 - **元组是不可变对象**, 但是有时候元组对象的值是可变的

```
s = 1, [2, [3, 4]], 5
# s[0] = 0
# TypeError: 'tuple' object does not support item assignment
s[1].append(0)
s[1][0] = -s[1][0]
print(s) # (1, [-2, [3, 4], 0], 5)
```

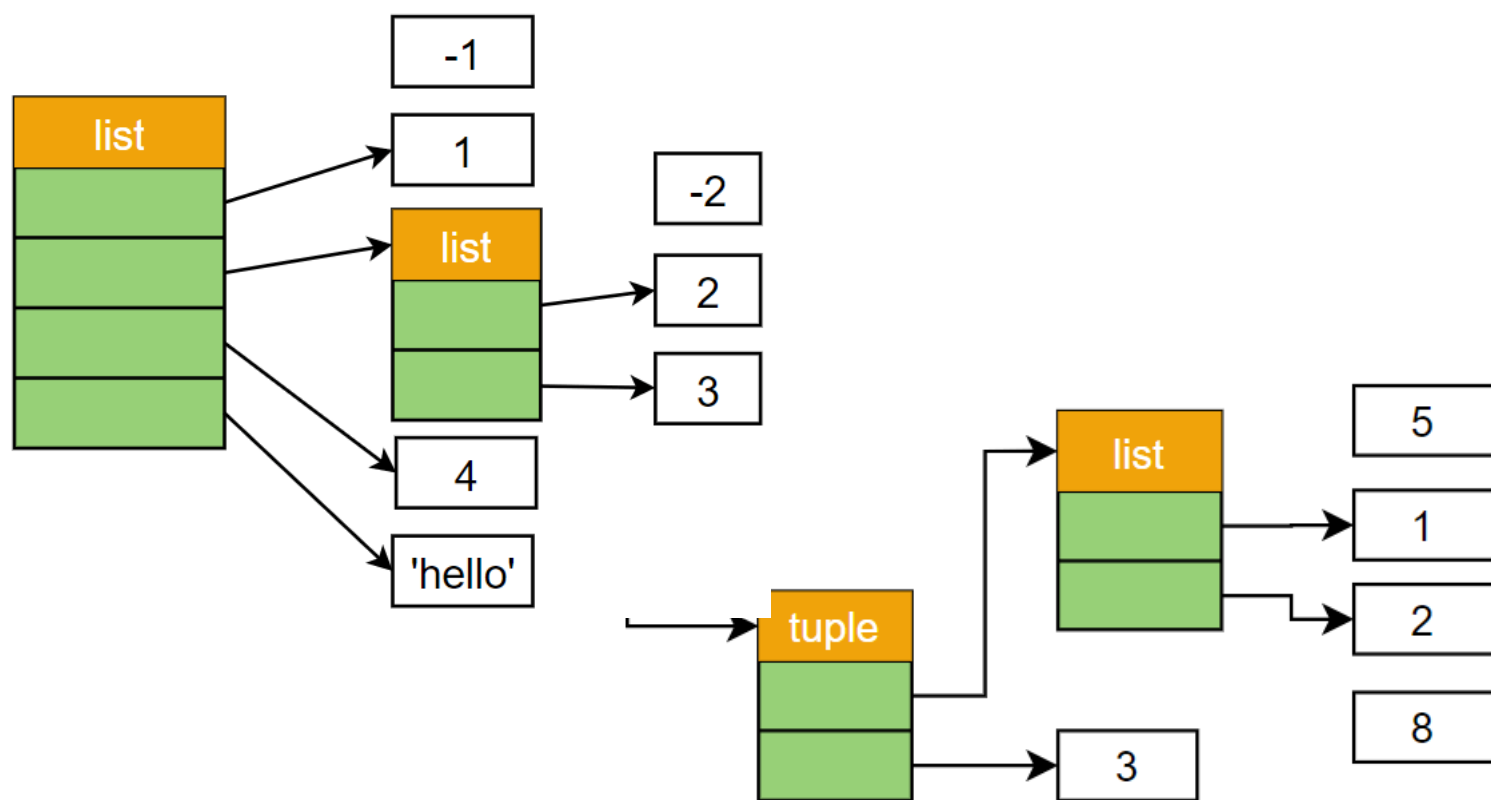
有序对象：下标形式

```
>>> d = [1, [2, 3], 4, 'hello']  
>>> len(d)  
4
```

```
>>> d[1][0] = -2    # d[1]是列表  
>>> d  
[1, [-2, 3], 4, 'hello']
```

```
>>> d[0] = -1      # d是列表
```

```
>>> d[3]  
'hello'  
>>> d[3][0] = 'H'  # d[3]是字符串  
... TypeError: 'str' object does  
not support item assignment
```



```
>>> x = ([1, 2], 3)
```

```
>>> x[0][0] = 5; x[0].append(8)    # x[0]是列表  
>>> x  
([5, 2, 8], 3)
```

```
>>> x[0] = x[0] + [10]             # x是元组  
TypeError: 'tuple' object does not support item assignment
```

构造函数法创建列表或元组

- 采用字面量方法创建一个列表或元组: [1, 2, 3] 或(1, 2, 3)
- 采用构造函数法创建列表或元组: list([iterable])或tuple([iterable])
 - list或tuple函数的参数可为空, 这时产生空列表或空元组
 - list或tuple函数的参数可为可迭代 (iterable) 对象(比如列表、range对象、字符串、元组、字典、集合等)
 - 创建一个新的列表或元组对象, 该对象的每个元素指向可迭代对象中各个元素指向的对象
- 在已有列表对象时, 还可通过该对象的copy()方法, 创建一个新的列表对象, 该对象的每个元素指向原列表对象的对应位置的元素。s.copy()等价于list(s)

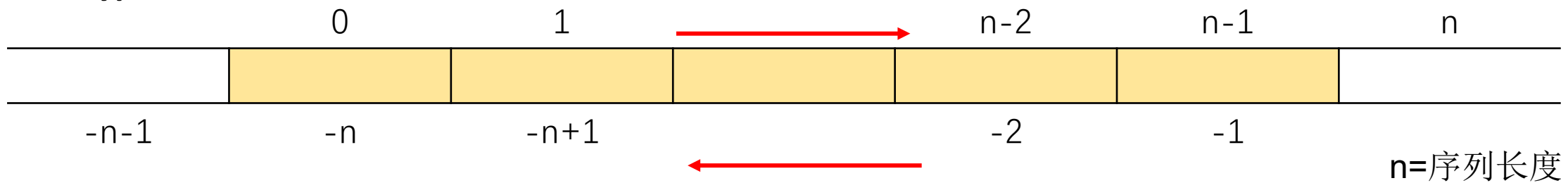
```
>>> x = list() # 创建空列表
>>> x
[]
>>> list([1, 3, 5, 7, 9])
[1, 3, 5, 7, 9]
>>> list(range(1, 10, 2))
[1, 3, 5, 7, 9]
```

```
>>> list((1, 3, 5, 7, 9))
[1, 3, 5, 7, 9]
>>> list('hello world')
['h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd']
>>> s = list(range(5))
>>> s.copy()
[0, 1, 2, 3, 4]
```

```
>>> tuple()
()
>>> tuple(range(5))
(0, 1, 2, 3, 4)
>>> tuple([1, 2, 3])
(1, 2, 3)
>>> tuple('123')
('1', '2', '3')
```

回顾：有序对象(列表、元组和字符串等) 的访问 【02-基本数据类型】

- seq[index]:使用**下标**访问列表元素，如果下标越界，抛出**IndexError**，如果下标不为整数或切片，抛出**TypeError**



- seq.index(value, [start, [stop]]) 获取指定值value在指定范围[start, stop]内**首次出现的下标**。缺省为从有序对象的第一个元素，到最后一个元素（包括）结束。也可以指定从start开始到stop(不包括)结束。如果无法找到值相等的元素，抛出异常**ValueError**

```
>>> a = [3, 4, 5, 7, 9, 11, 9, 7, 7, 6, 5]
>>> a[3]
7
>>> a.index(7)
3
>>> a.index(7, 4) # 从下标4开始查找
7
>>> a.index(7, 8, -1) # 从下标8开始，直到倒数第1个（不包括）为止
8
```

```
>>> a.index(13)
... ValueError: 13 is not in list
>>> t = (1, 2, 3, 2, 3)
>>> t.index(3)
2
```

回顾：有序对象(列表、元组和字符串等) 的访问 【02-基本数据类型】

- **seq.count(value)** 统计指定元素value(相等关系)在列表对象中出现的次数，不会抛出异常
- 成员关系判断运算符 in 和 not in:
 - value in seq 判断一个值是否存在于有序对象中(即是否有元素==value)，结果为True或False
 - value not in seq: 如果value不出现在有序对象中，结果为True，否则False
 - 运算符in也可用于其他容器对象，包括字典和集合等
- for循环中使用关键字in对可迭代对象的元素进行遍历，for obj in iterable: print(obj)

```
>>> a = [3, 4, 5, 7, 9, 11, 9, 7, 7, [6, 5]]
>>> a.count(7)
3
>>> a.count(0)
0
>>> 7 in a
True
>>> 7 not in a    # 不建议使用 not 7 in a
False
>>> 0 not in a
True
```

```
>>> [6, 5] in a
True
>>> t = (1, 2, 3, 2, 3)
>>> t.count(3)
2
>>> t.count(0)
0
>>> 3 in t
True
>>> 0 not in t
True
```

有序对象(列表、元组和字符串等) 的访问

- 异常处理的两种策略:
 - 避免出现异常: LBYL: Look Before You Leap
 - 异常出现后进行处理
 - EAFP: Easier to Ask for Forgiveness than Permission
- 自己实现index方法?
- 值在有序对象中可能多次出现, 如何返回所有出现位置的下标: 实现index_all方法

```
if 'zebra' in animals:  
    idx = animals.index('zebra')  
    print('zebra is at index {}'.format(idx))  
else:  
    print('zebra is not in the list')  
  
try:  
    idx = animals.index('zebra')  
    print('zebra is at index {}'.format(idx))  
except ValueError as e:  
    print('zebra is not in the list')
```

巧用成员关系判断运算符 in

`x == value1 or x == value2 or x == value3 or x == value4`

等价于: `x in (value1, value2, value3, value4)`

```
if x == 3 or x == 5 or x == 7:
    print('...')

if x in (3, 5, 7):
    print('...')

text = input('Please input...')
if text.lower() in ('bye', 'quit', 'exit'):
    print('bye!')
```

但是如果知道x为整数，且判断x是否为1到n的整数时:

- 不建议使用 `x in range(1, n+1)`
- 建议使用 `1 <= x <= n`

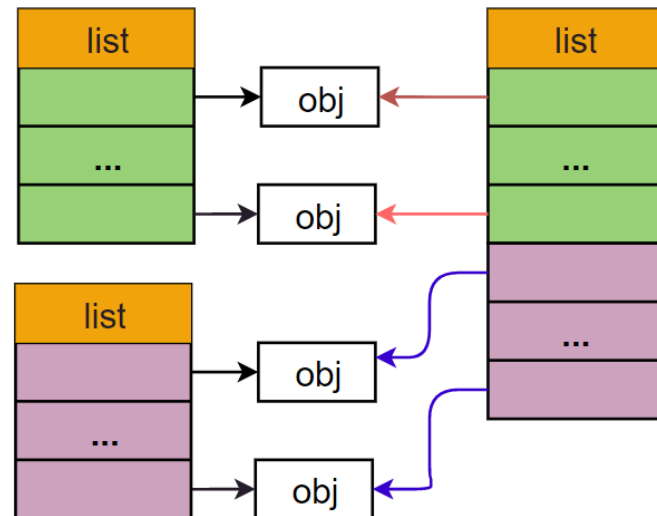
有序对象的加法和乘法运算

- 运算符 “+”

- 同种类型的有序对象相加，即列表+列表、元组+元组、字符串+字符串
- 创建一个新的同种类型的有序对象，由两个有序对象的元素按照先后顺序组成

- 运算符 “*”

- 有序对象(列表、元组、字符串等) 与整数相乘
- 生成一个新的有序对象，相当于原有序对象中的元素按照顺序重复出现多次
- 对于元组或列表而言，新的元组或列表中会有多个元素指向同一个对象(原元组或列表中的对应位置的元素所指向的对象)



```
>>> s = a = [3, 4, 5] + [7]
>>> a
[3, 4, 5, 7]
>>> a = a + [[7]]
>>> a
[3, 4, 5, 7, [7]]
>>> s is a
False
>>> a + 7
... TypeError: can only concatenate list (not "int") to list
>>> a = [1, 2, 3]
>>> a = a * 2
>>> a
[1, 2, 3, 1, 2, 3]
```

```
>>> a = (1, 2) + (3,)
>>> a
(1, 2, 3)
>>> a = a * 2
>>> a
(1, 2, 3, 1, 2, 3)
```

有序对象的比较运算：已经介绍过字符串的比较

- 关系运算符(大于、小于、大于等于、小于等于、等于、不等于) 可比较数值或序列的相等或大小
- 等于和不等于**几乎可以用于任何类型，而且不同类型的对象也可以比较相等关系
 - 相等：除了数值类型外，一般要求类型相同，且值相等
- 数字类型** (bool、整数、浮点等) 的对象之间可比较大小
- 列表/元组/字符串等有序序列的 (**同类**) 对象可以比较大小，按顺序比较各个元素的大小，如果前面比较确定了大小关系，则结束。如果其中某个对象没有对应的元素，则无元素表示其最小
- 字符之间的比较按照其在(Unicode)字符集中出现的先后顺序。 空格... 0...9 ...A...Z... a...z ...
- 算术运算符和关系运算符的优先级？ 算术运算符的优先级更高，即先算再比较

```
>>> 4 != 'ab'
True
>>> [1, 2] == (1, 2)
False
>>> [1, 2, 3] < [1, 2, 4]
True
>>> (1, 2, 3, 4) < (1, 2, 4)
True
```

```
>>> [1, 2] < [1, 2, -1]
True
>>> [1, 2, 3] == [1.0, 2.0, 3.0]
True
>>> [1, 2, ['aa', 'ab']] < [1, 2, ['abc', 'a'], 4]
True
>>> [1, 2] + [3] == [1, 2, 3]
True
```

主要内容

- 有序对象（字符串、range对象、列表和元组）的通用方法（第4章, 5.1, 5.2）
- **可变有序对象（列表）（第4章）**
- 有序对象的切片（第6章）
- 序列解包(2.4)
- 用于序列的常用内置函数（4.8）：zip和enumerate
- 函数式编程：
 - 列表推导式（6.5-6.7）
 - 生成器表达式
 - 高阶函数（12.6, 12.7）

列表对象的方法

方法		说明
增加	list.append(x)	将元素x添加至列表尾部
	list.extend(L)	将列表L中所有元素添加至列表尾部
	list.insert(index, x)	在列表指定位置index处添加元素x
删除	list.remove(x)	在列表中删除首次出现的指定元素
	list.pop([index])	删除并返回列表对象指定位置的元素
	list.clear()	删除列表中所有元素，但保留列表对象
查找	list.index(x)	返回值为x的首个元素的下标
	list.count(x)	返回指定元素x在列表中的出现次数
其他	list.reverse()	对列表元素进行原地逆序
	list.sort()	对列表元素进行原地排序
	list.copy()	返回列表对象的浅拷贝

del list_var
del list_var[index]

list_var[index]
x in list_var
x not in list_var

切片slicing: 获得列表中的多个元素, 增加、删除和修改列表中的元素

列表对象的方法：增加元素

- `append(object)`: 将`object`附加到列表对象，成为最后一个元素
- `extend(iterable)`：将另一个**可迭代对象**的所有元素添加至该列表对象尾部
- `insert(index, object)`：将元素添加至列表的指定位置，在原`index`对应的位置插入新元素，原来位置以及后面的元素相应往后移动一个位置。**`index`必须为`[-len(s), len(s)]`的整数**，否则throw `TypeError`
- **原地操作**列表，返回值为`None`

```
>>> a = a.append(5)
```

```
>>> print(a)
```

```
None
```

不要这样调用！！

`a = a.append(obj)`

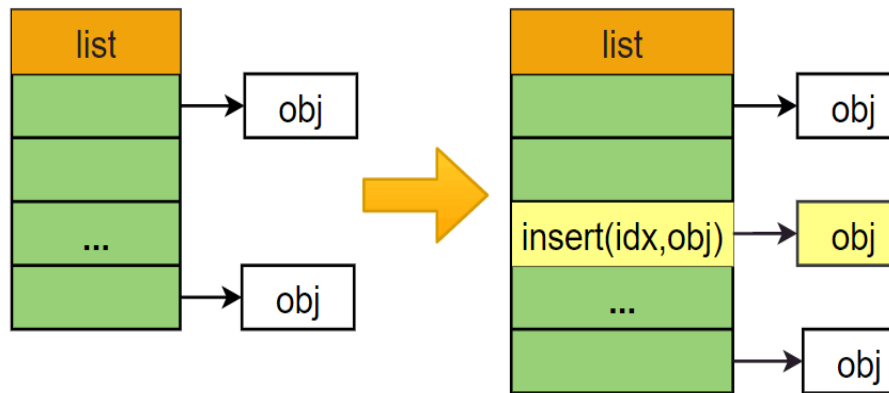
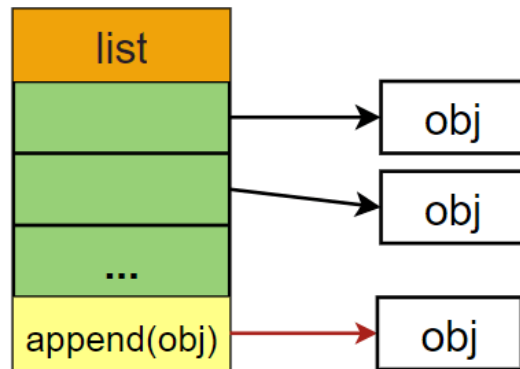
`a = a.extend(iterable)`

`a = a.insert(idx, obj)`

```
>>> a = [3, 4, 5, 7]
>>> a.append(9)
>>> a.append([10])
>>> a
[3, 4, 5, 7, 9, [10]]
```

```
>>> a = [1, 2]
>>> a.extend(1)
... TypeError: 'int' object
is not iterable
>>> a.extend([3, 4])
>>> a.extend([[5, 6]])
>>> a
[1, 2, 3, 4, [5, 6]]
>>> a.extend(range(2))
>>> a
[1, 2, 3, 4, [5, 6], 0, 1]
```

```
>>> a = [3, 4, 5, 7, 9]
>>> a.insert(3, 6)
>>> a
[3, 4, 5, 6, 7, 9]
>>> a.insert(0, 2) # 最前面插入
>>> a
[2, 3, 4, 5, 6, 7, 9]
>>> a.insert(len(a), -1) # 尾部附加
>>> a
[2, 3, 4, 5, 6, 7, 9, -1]
>>> a.insert(-1, -2) # -2 成为倒数第二个
>>> a
[2, 3, 4, 5, 6, 7, 9, -2, -1]
```



列表对象的方法：增加元素

【不要求】复合赋值语句

`list_obj += iterable` 等价于 `list_obj.extend(iterable)`, **原地附加**多个元素

`list_obj *= 4` 与*运算类似, 只是**原地重复**, 重复元素实际上是多个元素**指向同一对象**

```
>>> s1 = list(range(4))
>>> alias = s1
>>> s1 += [4, 5]
>>> s1 += 'ok'
>>> s1
[0, 1, 2, 3, 4, 5, 'o', 'k']
>>> s1 is alias    # 说明 += 为原地操作
True

>>> s1 *= 3
>>> s1
[0, 1, 2, 3, 4, 5, 'o', 'k', 0, 1, 2, 3, 4, 5, 'o', 'k', 0, 1, 2, 3, 4, 5, 'o', 'k']
>>> s1 is alias    # 说明 *= 为原地操作
True
```

列表对象的方法：增加元素

#	列表元素增加方法	别名	效果
1	append(obj)	附加	原地修改
2	extend(iterable)	扩展	原地修改
3	insert(index, obj)	插入	原地修改
4	list1 += iterable	复合赋值	原地修改
5	list1 *= n	复合赋值	原地重复
6	list1 + list2	拼接	新建列表
7	list1 * n	复制	新建列表

注意：

insert()方法会涉及到插入位置之后所有元素的移动，这会影响处理速度

建议：除非有必要，否则应尽量避免在列表中间位置插入元素的操作，而是优先考虑使用append()等方法在尾部附加

求n以内的平方数

square_numbers.py

```
import math
def square_numbers(n=100):
    numbers = []
    for i in range(n+1):
        k = int(math.sqrt(i))
        if k * k == i:
            numbers.append(i)
    return numbers
```

```
def square_numbers2(n=100):
    numbers = []
    for i in range(int(math.sqrt(n)) + 1):
        numbers.append(i*i)
    return numbers
```

[0**2, 1**2, ..., int(sqrt(n)) ** 2]

列表对象的方法：增加元素示例

13-13：大于2的正偶数分解成两个素数之和

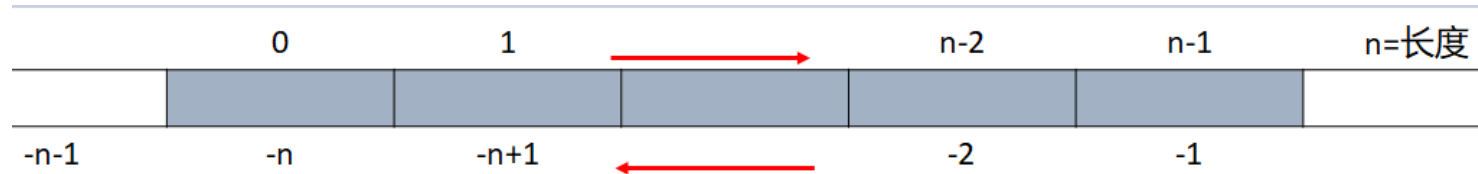
```
def is_prime(n):
    max_factor = int(math.sqrt(n) + 1)
    for factor in range(2, max_factor):
        if n % factor == 0:
            return False
    return True

def decompose(n):
    if not isinstance(n, int) or n < 2 or n % 2:
        raise ValueError("n must be an even integer larger than 2")
    pairs = []
    for i in range(2, n // 2 + 1):
        if is_prime(i) and is_prime(n - i):
            pairs.append((i, n - i))
    return pairs
```

decompose.py

```
not (isinstance(n, int) and n >= 2 and n % 2 == 0)
```

列表对象的方法：删除元素



- 删除指定位置的元素：del seq[idx]
 - 下标越界时，抛出**异常IndexError: list assignment index out of range**
 - 下标不为整数或切片对象时，**抛出异常 TypeError: list indices must be integers or slices, not ...**
- 删除指定位置idx(缺省为-1，即最后一个)的元素，并且得到该元素：seq.pop(idx=-1)
- 根据值删除第一个出现（即值相等）的元素：seq.remove(value)
 - 如果列表中有元素的值与value的**值相等**（seq[index] == value），则删除第一个值相等的元素，**返回None**。如果列表中不存在要删除的元素，则抛出**异常ValueError**

```
>>> a = list(range(3, 16, 2))
>>> a
[3, 5, 7, 9, 11, 13, 15]
>>> del a[1], a[-1]
>>> a
[3, 7, 9, 11, 13]
>>> del a[5]
... IndexError: list assignment index out of range
>>> a.pop()
13
>>> a.pop(-1)
11
>>> a.pop(1)
7
```

```
>>> a = [3, 5, 7, 9, 7, [4, 5]]
>>> a.remove(7)
>>> a
[3, 5, 9, 7, [4, 5]]
>>> a.remove(7)
>>> a
[3, 5, 9, [4, 5]]
>>> a.remove(7)
... ValueError: list.remove(x): x not in list
>>> a.remove([4, 5])
>>> a
[3, 5, 9]
```

列表对象的方法：删除元素

- 列表的clear()方法，删除列表对象的所有元素，即变为空列表

```
>>> s = [1, 2, 3]
>>> s.clear()
>>> s
[]
```

方法	作用	返回	备注
del list_obj[index]	删除指定位置元素	无返回值	IndexError,TypeError
pop([index])	删除指定位置元素并返回该元素	返回元素	IndexError,TypeError
remove(value)	删除首次出现的指定元素, 返回None	返回None	没有相应元素时ValueError
clear()	删除所有元素	返回None	空列表

注意：

- insert()方法会涉及到插入位置之后所有元素的移动，这会影响处理速度
- 列表删除方法pop()弹出非尾部元素时，也有类似问题

建议：除非有必要，否则应尽量避免在列表中间或头部位置位置插入和删除元素的操作，而是优先考虑使用append()方法尾部附加和pop()方法尾部删除

回顾：random模块

```
>>> s = random.choices(range(5), k=7)
>>> s
[1, 0, 1, 0, 3, 1, 1]
>>> random.sample(s, 4)
[1, 0, 0, 1]
```

```
import random
# [1, 100)之间随机选择4个不同的整数
print(random.sample(range(1, 100), 4))
print(random.choices(range(1, 100), k=4)) # 可能相同
print(random.choices(range(4), (1,1,2,2), k=2))
```

seed(a=None)	给出随机数生成器的seed
randint(a,b)	返回在[a,b]区间的随机整数
randrange(start, stop=None, step=1)	从range(start, stop[, step])中随机选择一个整数
random()	返回在[0,1)区间的随机实数
uniform(a,b)	返回在[a,b)区间的随机实数
choice(seq)	返回非空有序序列seq中的（随机选择的）某个元素
choices(seq, weights=None, *, k=1)	从非空有序序列seq中随机(返回)选择k个元素组成一个新列表。weights可传递seq中每个元素的权重，否则每个元素被选中的机率相同
sample(seq, k, *, counts=None)	从非空有序序列seq随机(不放回)选择 k个元素组成一个新列表
shuffle(listobj)	列表listobj原地随机排序，返回None

列表的排序

(1) 使用列表对象的**sort方法**进行**原地**排序。 `help(list.sort)`

L.sort(key=None, reverse=False) -> 返回None，默认基于元素间的大小关系排序，升序排列

- `reverse`缺省为`False`，表示升序，即从小到大；如果传递参数`reverse=True`时为降序排列
- `key`缺省为`None`，表示基于元素间的大小关系排序。也可传递函数对象，作为排序的基准

(2) 使用**内置函数sorted**对可迭代对象(列表/字符串/元组等)进行排序并**返回新列表**

sorted(iterable, key=None, reverse=False)

```
>>> s = [3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> import random
>>> random.shuffle(s)
>>> s
[13, 5, 6, 11, 9, 7, 15, 4, 3, 17]
>>> s1 = list(s)

>>> s.sort() # 默认为升序排列
>>> s
[3, 4, 5, 6, 7, 9, 11, 13, 15, 17]
>>> s1.sort(reverse=True) # 降序排列
>>> s1
[17, 15, 13, 11, 9, 7, 6, 5, 4, 3]
```

```
>>> s = [10, 5, 23, 20]
>>> s1 = sorted(s)
>>> s1
[5, 10, 20, 23]

>>> s2 = sorted(s, reverse=True)
>>> s2
[23, 20, 10, 5]

>>> sorted('python')
['h', 'n', 'o', 'p', 't', 'y']
```

列表的逆序

- 使用列表对象的**reverse方法**将元素原地逆序(即列表中元素出现的相反顺序) , 注意不等同于 L.sort(reverse=True)
- 使用**内置函数reversed**对可迭代对象(如列表)的元素进行逆序排列并返回**reversed对象**
 - sorted函数返回新的列表
 - reversed函数返回的不是列表, 而是一个可迭代对象, 更是一个**迭代器**

```
t = [3, 4, 6, 7, 5, 9, 11, 17, 13, 15]
t.reverse()
print(t)
# [15, 13, 17, 11, 9, 5, 7, 6, 4, 3]
```

```
右边代码的输出:
<class 'list_reverseiterator'>
[3, 4, 6, 7, 5, 9, 11, 17, 13, 15]
-----
3, 4, 6, 7, 5, 9, 11, 17, 13, 15,
```

```
m = reversed(t)
print(type(m))
print(list(m))

for item in m:      # 没有输出
    print(item)

print('-' * 40)

for item in reversed(t):
    print(item, end=', ')
```

总结：列表对象的方法

方法		说明
增加	list.append(x)	将元素x添加至列表尾部
	list.extend(L)	将列表L中所有元素添加至列表尾部
	list.insert(index, x)	在列表指定位置index处添加元素x
删除	list.remove(x)	在列表中删除首次出现的指定元素
	list.pop([index])	删除并返回列表对象指定位置的元素
	list.clear()	删除列表中所有元素，但保留列表对象
查找	list.index(x)	返回值为x的首个元素的下标
	list.count(x)	返回指定元素x在列表中的出现次数
其他	list.reverse()	对列表元素进行原地逆序
	list.sort()	对列表元素进行原地排序
	list.copy()	返回列表对象的浅拷贝

del list_var
del list_var[index]

list_var[index]
x in list_var
x not in list_var

切片slicing： 获得列表中的多个元素, 增加、删除和修改列表中的元素

列表元素的删除+for循环（第8章循环结构：教材P164）

例8-12 如果要删除列表中所有出现的某个元素怎么办？for循环遍历，发现元素时删除该元素

```
def remove_all(list_, value):  
    for item in list_:  
        if item == value:  
            list_.remove(value)  
    return list_
```

remove_all.py

```
def remove_all_0(list_, value):  
    for i in range(len(list_)):  
        print(i)  
        if list_[i] == value:  
            del list_[i]  
    return list_
```

```
x = [1,2,1,2,1,2,1,2,1]  
print(remove_all(x,1))  
# [2, 2, 2, 2]  
x = [1,2,1,2,1,1,1]  
print(remove_all(x,1))  
# [2, 2, 1] ???
```

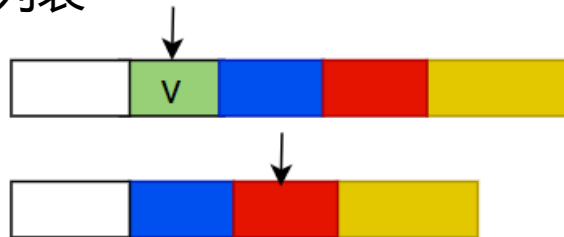
列表的迭代器实现：

- 一般假设原有的可迭代对象并不会改变，至少下标不变
- 迭代器：维护当前下标pos
- 如果pos < len(x)，则下一个元素为x[pos]; pos += 1

pos = 0	[1,2,1,2,1,1,1]	i = x[pos] → 1	remove
pos = 1	[2,1,2,1,1,1]	i = x[pos] → 1	remove
pos = 2	[2,2,1,1,1]	i = x[pos] → 1	remove
pos = 3	[2,2,1,1]	i = x[pos] → 1	remove
pos = 4	[2,2,1]	pos >= len(x)	stop

例子中：一旦remove(value)或del list_[i]，后面的元素往前移动一个位置，导致会跳过后一个元素！

每轮循环从列表中取下一个元素，但循环体中修改了列表



基本观察：

1. 每当插入或删除一个元素之后，该元素位置后面所有元素的索引就都改变了
2. 迭代器实现一般假设原有的可迭代对象并不会改变，至少下标不变

列表元素的删除+for循环

x = [1,2,1,2,1,1,1]

复制 xx = [1,2,1,2,1,1,1]

```
def remove_all_2(list_, value):  
    for item in list_.copy(): # list_  
        if item == value:  
            list_.remove(value)  
    return list_
```

remove_all.py

```
def remove_all_3(list_, value):  
    for i in range(len(list_) - 1, -1, -1):  
        if list_[i] == value:  
            del list_[i]  
    return list_
```

注意while的条件表达式每轮循环都会重新计算，但是for循环in后面的表达式只会计算一次

it_ = iter(list_[:])

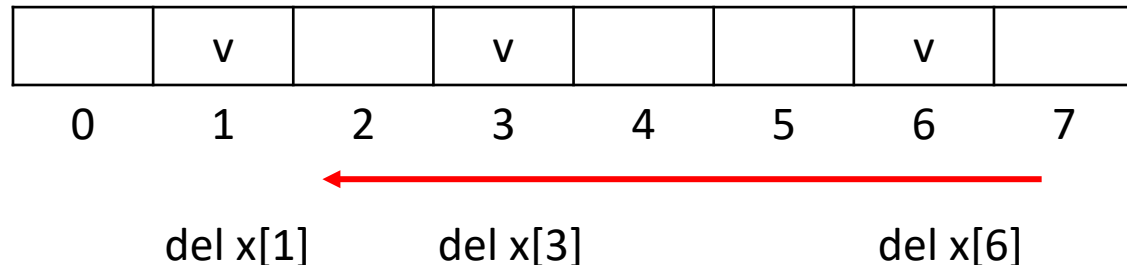
loop item = next(it_)

基本观察:

1. 每当插入或删除一个元素之后，该元素位置后面所有元素的索引就都改变了
2. 迭代器实现一般假设原有的可迭代对象并不会改变，至少下标不变

怎么办?

1. for循环中，可迭代对象不是原始的列表，而是使用**克隆的列表（如列表的切片）替代**，即使原始列表因元素删除或增加而变化，克隆的列表不会变化
2. 使用正确的顺序，例如从后往前依次判断。**通过下标**从后面删除时，被删除元素之前的元素的下标不会改变
3. 不使用for循环，而是采用while，并且在循环体中考虑下标或索引改变的情况



列表元素的删除+for循环

```
def remove_all_4(list_, value):  
    i = 0  
    while i < len(list_):  
        if list_[i] == value:  
            del list_[i] #不需要更新i  
        else:  
            i = i + 1  
    return list_
```

remove_all.py

- 初始化 $i = 0$ ，表示从头开始检查
- 循环体中：
 - 正常情况下 $i = i + 1$ ，检查下一个元素
 - 当前元素为 $value$ 时，移走该 $value$ ，下一个元素的下标仍然是 i

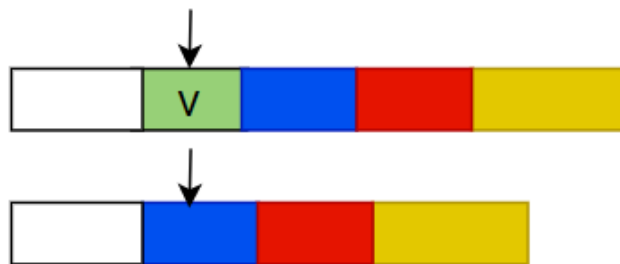
	v		v			v	
0	1	2	3	4	5	6	7

基本观察:

1. 每当插入或删除一个元素之后，该元素位置后面所有元素的索引就都改变了
2. 迭代器实现一般假设原有的可迭代对象并不会改变，至少下标不变

怎么办?

1. for循环中，可迭代对象不是原始的列表，而是使用**克隆的列表（如列表的切片）替代**，即使原始列表因元素删除或增加而变化，克隆的列表不会变化
2. 使用正确的顺序，例如从后往前依次判断。**通过下标**从后面删除时，被删除元素之前的元素的下标不会改变
3. 不使用for循环，而是**采用while**，并且在循环体中考虑下标或索引改变的情况



主要内容

- 有序对象（字符串、range对象、列表和元组）的通用方法（第4章, 5.1, 5.2）
- 可变有序对象（列表）（第4章）
- **有序对象的切片（第6章）**
- 序列解包(2.4)
- 用于序列的常用内置函数（4.8）：zip和enumerate
- 函数式编程：
 - 列表推导式（6.5-6.7）
 - 生成器表达式
 - 高阶函数（12.6, 12.7）

有序对象的切片(slice)

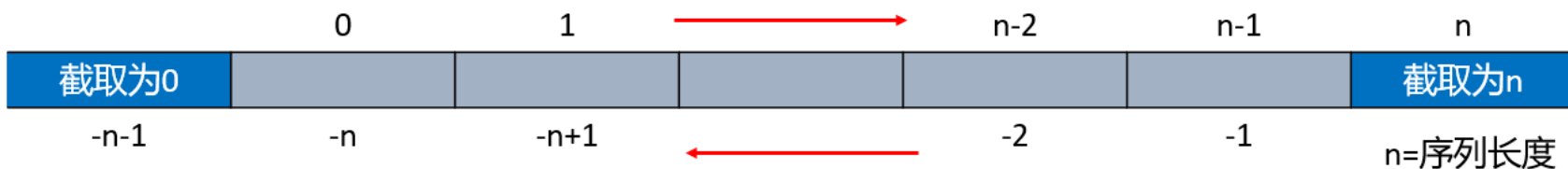
- 可通过**整数下标**来访问有序对象对应位置的元素
- 可通过**整数下标与赋值语句**结合修改列表中对应位置的元素
- 可通过**整数下标与del语句**结合删除列表中对应该位置的元素
- 整数下标(指定的某个位置) → 切片下标 (指定的多个位置)
 - 可通过**切片下标**访问有序对象中多个位置的元素
 - 可通过**切片下标与赋值语句**结合修改和删除列表中多个位置的元素
 - 可通过**切片下标与del语句**结合删除列表中多个位置的元素
- 切片格式:
 - start:stop 从start开始到stop为止 (不包括) 的元素
 - start:stop:step: 指定步长, 而不是缺省的连续元素

```
>>> s = list(range(10))
>>> s
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> s[-1]
9
>>> s[-1] = -s[-1]
>>> s
[0, 1, 2, 3, 4, 5, 6, 7, 8, -9]
>>> s[0:4]
[0, 1, 2, 3]
>>> s[0:4] = [0] * 4
>>> s
[0, 0, 0, 0, 4, 5, 6, 7, 8, -9]
>>> del s[-2:]
[0, 0, 0, 0, 4, 5, 6, 7]
```

有序对象的切片(slice): 基本切片

`seq[start:stop]`: 有序对象`seq`中那些下标在`[start,stop)`的元素 (可0个), 组成一个新的同种类型的有序对象

- 从左往右方向选取对应位置的元素
- `start`和`stop`都可省略, 表示采用缺省值。如`seq[:5]` `seq[5:]` `seq[:]`
- `start`缺省为0, `stop`缺省为**列表结束即`len(s)`**
- `start < stop` (下标意义上的比较, 而不是简单的整数大小比较) 才可访问到相应的元素, 否则对应的元素序列为空
 - 合法的下标范围为 `[-len(seq), len(seq))`
 - 下标意义比较: 如果`idx < 0`, 可转换为`idx + len(seq)`
 - **切片的下标超出范围时, 会截取到合适的下标**
 - 如果`< 0`, 则相当于0
 - 如果`> len(seq)`, 则相当于`len(seq)`



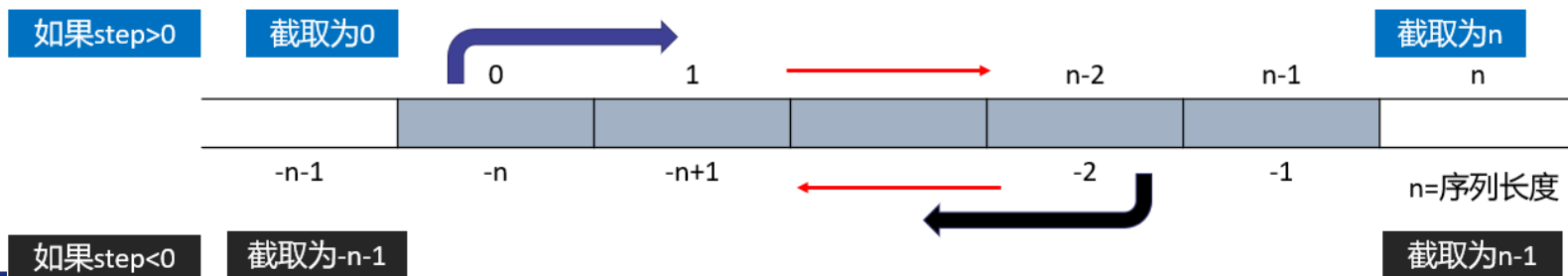
```
>>> s = list(range(10))
>>> s
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> s[3:6]
[3, 4, 5]
>>> s[:5]
[0, 1, 2, 3, 4]
>>> s[5:]
[5, 6, 7, 8, 9]
>>> s[:]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> s[1:1]
[]
>>> s[1:-1]
[1, 2, 3, 4, 5, 6, 7, 8]
>>> s[6:20]
[6, 7, 8, 9]
>>> s[-20:3]
[0, 1, 2]
>>> s[20:3]
[]
>>> s[1:-20]
[]
>>> t = tuple(range(4))
>>> t
(0, 1, 2, 3)
>>> t[1:-1]
(1, 2)
>>> m = 'abcdefg'
>>> m[1:-1]
'bcdef'
```

有序对象的切片(slice): 扩展切片

`seq[start:stop:step]`: `step`可以省略, 表示缺省为1。描述了有序对象中从`start`开始, 每隔`step`的元素, 直到`stop` (不包括) 为止

- `step`可以为负值, 表示从有序对象中`start`开始, 从后面往前访问相应位置的元素, 直到`stop` (不包括) 为止。这些元素之间相隔指定间隔`step`
- 合法下标范围为 $[-\text{len}(s), \text{len}(s))$, 下标超出范围时, 截取到合适的下标
- 下标意义比较: 如果 $\text{idx} < 0$, 可转换为 $\text{idx} + \text{len}(\text{seq})$
- 如果 $\text{step} > 0$, 则`start`缺省为列表开头(`start=0`), `stop`缺省为列表长度 (即最后一个元素再后一个位置), `stop=len(seq)`
 - $> \text{len}(\text{seq})$ 时, 相当于 $\text{len}(\text{seq})$
 - < 0 时, 相当于0
- 如果 $\text{step} < 0$, 则`start`缺省为列表最后一个元素, `start=len(seq)-1`。 `stop`缺省为列表第一个元素再前一个位置, 即`stop=-len(seq) - 1`
 - $> \text{len}(\text{seq})-1$ 时, 相当于 $\text{len}(\text{seq})-1$
 - < 0 时, 相当于 $-\text{len}(\text{seq}) - 1$

```
>>> s = list(range(10))
>>> s
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> s[::2]
[0, 2, 4, 6, 8]
>>> s[1::2]
[1, 3, 5, 7, 9]
>>> s[2:-2:2]
[2, 4, 6]
>>> s[::-1]
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
>>> s[-1:-4:-1]
[9, 8, 7]
>>> s[-1::-2]
[9, 7, 5, 3, 1]
>>> s[-2::-2]
[8, 6, 4, 2, 0]
```



有序对象的切片(slice):切片的使用

- **出现在表达式**中时:

- 下标为整数时, `seq[idx]` 表示访问有序对象中对应位置的元素
- 下标为切片时, `seq[slice]`表示访问有序对象中多个位置的元素, 最终得到的是由那些位置的元素组成的新的同种类型的有序对象
- 注意: 切片得到的新的有序对象的元素与原有序对象的对应元素指向同一个对象, 即浅拷贝 (shallow copy), 后续会展开

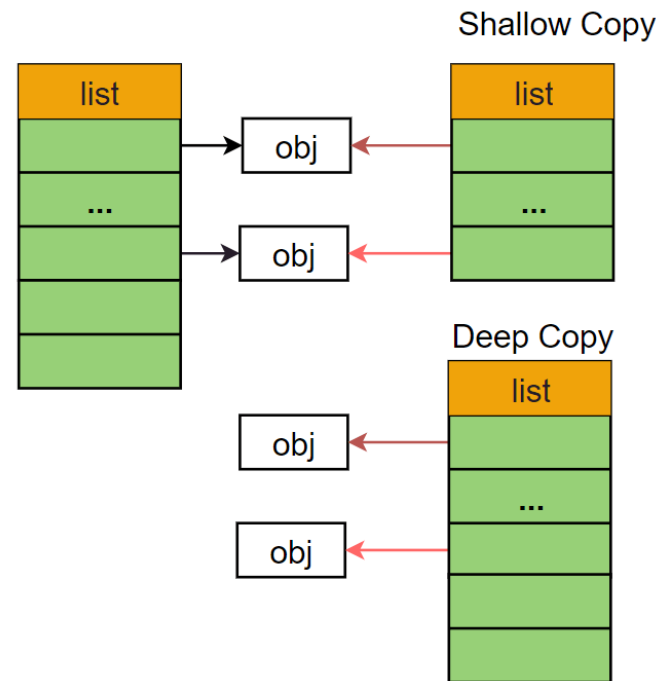
- 如果有序对象是可变的, 即有序对象seq为列表时:

- **下标与del语句结合:**

- 下标为整数时, `del seq[idx]`表示删除列表中对应该位置的元素
- 下标为切片时, `del seq[slice]`表示删除列表中多个对应位置的元素

- **下标与赋值语句结合:**

- 下标为整数时, `seq[idx] = value`, 表示列表对应位置的元素改变为value
- 下标为切片时, `seq[slice] = iterable`, 表示改变 (修改或删除) 列表多个位置的元素



```
>>> s = [[1, 2], [3, 4]]
>>> s1 = s[:]
>>> s1[0] is s[0]
True
```

使用下标访问时, 下标越界会抛出异常。切片操作则不会, **下标超出范围时, 仅返回能遍历到的元素或空序列**, 代码具有更强的健壮性

通过切片修改列表

使用切片原地修改列表内容：添加多个新元素

`seq[i:i] = iterable`

赋值语句左边为列表的切片形式，且start/stop为同一个位置，右边为iterable对象，即在列表对应的位置i处插入iterable对象中的各个元素

- 可以在尾部添加多个元素: `seq[len(seq):]`
- 可以在头部插入多个元素: `seq[0:0]`
- 可以在中间(位置i)插入多个元素: `seq[i:i]`

使用切片原地修改列表内容, 替换或者删除元素

`seq[start:stop] = iterable`

`seq[start:stop:step] = iterable`

赋值语句左边的切片下标描述的位置**不为空**，相当于将指定位置的元素**替换为**赋值语句右边iterable对象中的各个元素

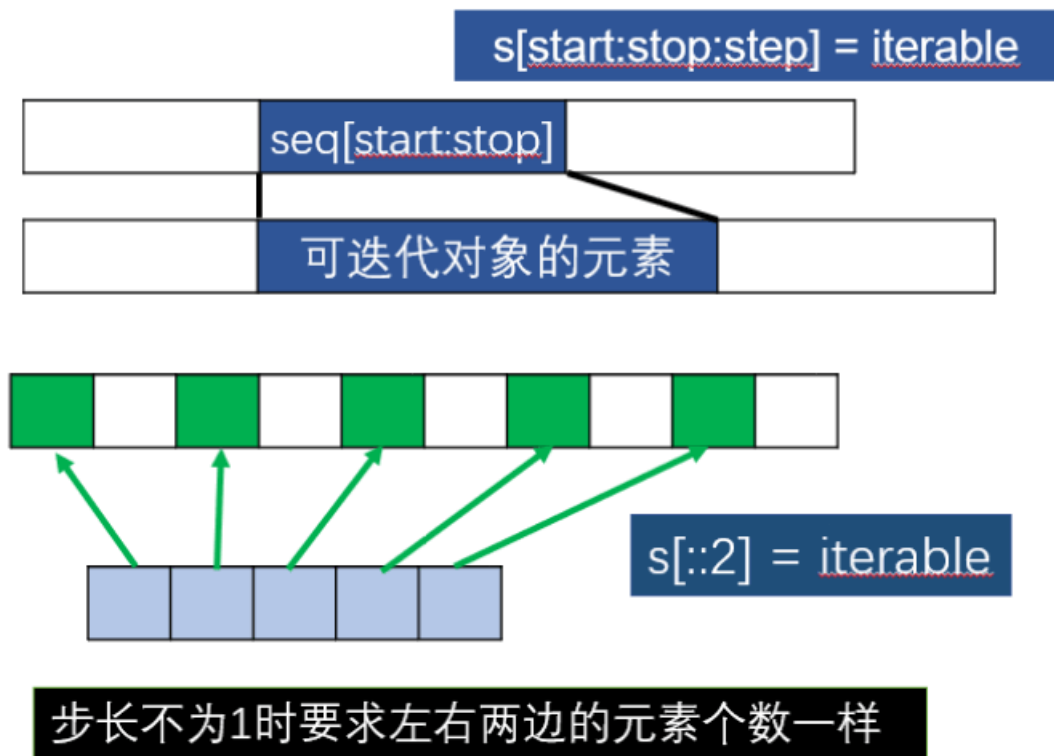
- 右边iterable对象长度为0，比如为空列表时，相当于删除对应的元素
- step为1时，即连续位置，不要求两边元素个数一样

```
>>> s = [3, 5, 7]
>>> s[len(s):] = [9, 11] #尾部
>>> s
[3, 5, 7, 9, 11]
>>> s[0:0] = [-1, 1]    #头部
>>> s
[-1, 1, 3, 5, 7, 9, 11]
>>> s[3:3] = [51, 53]   #指定位置
>>> s
[-1, 1, 3, 51, 53, 5, 7, 9, 11]
>>> s[-1:-1] = [10]
>>> s
[-1, 1, 3, 51, 53, 5, 7, 9, 10, 11]
>>> s[:3] = () # 删除前面3个元素
>>> s
[51, 53, 5, 7, 9, 10, 11]
>>> s[:3] = [1, 2] # 替代前面3个元素
>>> s
[1, 2, 7, 9, 10, 11]
>>> s[:3] = range(4)
>>> s
[0, 1, 2, 3, 9, 10, 11]
```

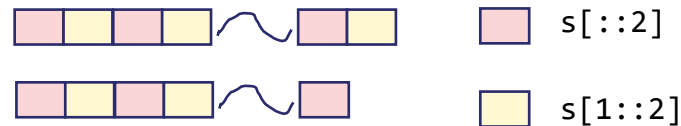


通过切片修改列表

- 列表的切片和赋值语句结合时，**切片步长不为1(即便为-1) 时**，要求左右两边的元素个数一样
 - 切片和赋值语句结合只能删除连续多个元素
- 列表的切片和del语句结合： `del seq[slice]`
 - 对于slice中的步长没有限制



```
>>> s = list(range(9))
>>> s
[0, 1, 2, 3, 4, 5, 6, 7, 8]
>>> s[::2] = [0] * ((len(s) + 1) // 2)
>>> s
[0, 1, 0, 3, 0, 5, 0, 7, 0]
>>> s[1::2] = [1] * (len(s) // 2)
>>> s
[0, 1, 0, 1, 0, 1, 0, 1, 0]
>>> s[::2] = []
>>> s
[0, 1, 0, 1, 0, 1, 0]
>>> s[::2] = []
...
ValueError: attempt to assign sequence
of size 0 to extended slice of size 4
>>> del s[::2]
>>> s
[1, 1, 1]
```



切片示例1: 字符串的join方法

问题6-1 筛选字符串。筛选出以ly结尾的单词，并统计其数量

```
def count_ly_words(words):  
    ly_words = []  
    for word in words:  
        if word[-2:] == "ly":  
            ly_words.append(word)  
  
#     print("共有%d个单词以ly结尾:" % len(ly_words))  
    print(f"共有{len(ly_words)}个单词以ly结尾")  
    print(", ".join(ly_words))
```

count_ly_words.py

字符串的join方法: 组合多个字符串, `str.join(iterable)`

- 将iterable对象(列表,字符串等)中的多个元素组合在一起, 元素间插入相应的**分割字符串strobj**, 返回新的字符串
- iterable对象中的**元素必须是字符串**

```
>>> s = list('abc')  
>>> s  
['a', 'b', 'c']  
>>> ''.join(s)  
'abc'  
>>> ', '.join(s)  
'a,b,c'  
>>> print('\n'.join(s))  
a  
b  
c
```

切片示例2

问题6-2:用户随意输入一串字符，程序依次在相邻位置插入一个空格，逐行输出。

- 将word分割为两个部分，之间插入空格。观察分割处的位置变化
 - 从最后一个位置开始到第2个位置（包括）
 - 第2个位置开始到最后一个位置（包括）为止

Enter a word:

python

pytho n

pyth on

pyt hon

py thon

p ython

p ython

py thon

pyt hon

pyth on

pytho n

```
def insert_space(word):
    if len(word) <= 1:
        raise ValueError("Input must have more than one character.")
    for i in range(len(word) - 1, 0, -1):
        print(word[:i] + " " + word[i:])
    print("-" * (len(word) + 1))
    for i in range(1, len(word)):
        print(word[:i] + " " + word[i:])

if __name__ == "__main__":
    text = input("Enter a word: ")
    insert_space(text)
```

insert_space.py

切片示例3

问题6-3: 列表中的元素**循环左移k位**, 返回新的列表。假设原列表为: `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`, 循环左移k位, 则列表为: `[6, 7, 8, 9, 10, 1, 2, 3, 4, 5]`

问题6-4: 原地循环左移k位, 不生成新的列表

逆序

- `s.reverse()`: 原地逆序
- `s[::-1]`: 切片逆序得到列表
- `reversed(s)`: 迭代器对象



A1A2...Ak	B1B2...Bn
Ak...A2A1	Bn...B2B1
B1B2...Bn	A1A2...Ak

问题6-4变种: 原地循环左移k位, 不生成新的列表, 也不允许引入其他变量保存列表的部分内容

- 算法思路: 将s中下标k之前 (不包括) 的元素逆序, 下标k及其后的元素逆序, 最后将整个列表逆序

```
def left_shift_list(s, offset):  
    offset = offset % len(s)  
    return s[offset:] + s[:offset]
```

引入中间变量

```
def list_left_shift(s, offset):  
    offset = offset % len(s)  
    shift_part = s[:offset]  
    del s[:offset]  
    s.extend(shift_part)  
    # s[len(s):] = shift_part
```

```
def list_left_shift1(s, offset):  
    offset = offset % len(s)  
    s[:offset] = s[offset - 1 :: -1]  
    s[offset:] = s[: offset - 1 : -1]  
    s.reverse()
```

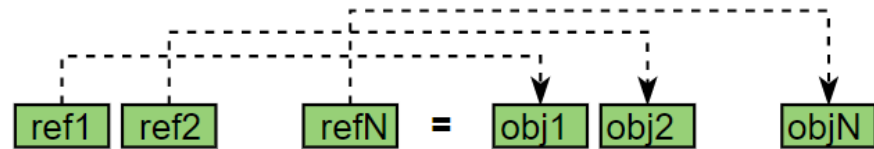
```
def list_left_shift2(s, offset):  
    offset = offset % len(s)  
    s[:offset] = reversed(s[:offset])  
    s[offset:] = reversed(s[offset:])  
    s.reverse()
```

lshift.py

主要内容

- 有序对象（字符串、range对象、列表和元组）的通用方法（第4章, 5.1, 5.2）
- 可变有序对象（列表）（第4章）
- 有序对象的切片（第6章）
- **序列解包（2.4）**
- 用于序列的常用内置函数（4.8）：zip和enumerate
- 函数式编程：
 - 列表推导式（6.5-6.7）
 - 生成器表达式
 - 高阶函数（12.6, 12.7）

序列解包(sequence unpacking)



- 序列解包也称为iterable unpacking, 将**可迭代对象**拆分成各个元素, 赋值给多个**对象引用**(如变量)
 - 对象引用可以是**变量名**, 可以是**属性**, 可以通过**下标**描述的容器对象中的元素, 可以通过**切片**描述的列表中的元素
- 赋值语句与序列解包: LHS=RHS, 左边LHS为通过**元组或列表形式描述的多个变量引用**
 - LHS通过圆括号和方括号(不引起歧义时可省略) 组织对象引用, 通过逗号分割引用
 - RHS可以是任何可迭代对象, 包括tuple、list、dict、range、str等, 逐个按顺序 (从左到右) 取该可迭代对象的元素赋予左边对应位置的对象引用
 - **基本序列解包**, 要求左边LHS中的对象引用个数与RHS中的元素**个数相同**
 - **扩展序列解包**, 允许RHS的元素个数**大于等于**左边对象引用的个数, 可使用**带星号的对象引用** (*seq) 来收集RHS中的**多余元素**

```
>>> x, y, z = (False, 3.5, 'exp')
>>> x, y, z
(False, 3.5, 'exp')
>>> a, b, c = [1, 2, 3]
>>> a, b, c
(1, 2, 3)
```

```
>>> (a, b, c) = [1, 2, 3]
>>> a, b, c
(1, 2, 3)
>>> [a, b, c] = 'abc'
>>> a, b, c
('a', 'b', 'c')
```

```
>>> a, b, c = range(3)
>>> a, b, c
(0, 1, 2)
>>> a, b, c = range(4)
Traceback (most recent call
last):
ValueError: too many values to
unpack (expected 3)
```

序列解包(sequence unpacking)

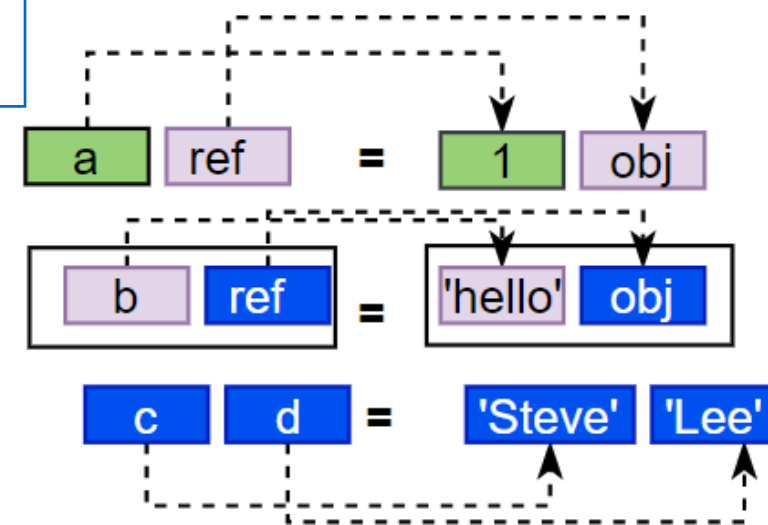
- 对象引用可以是变量名，可以是属性，可以通过下标描述的容器对象中的元素，可以通过切片描述列表中的元素
- 对象引用为**切片**时，必须与**可迭代对象对应**

```
>>> import math
>>> s = list(range(12))
# s = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
>>> x, math.y, s[-1], s[0:5] = 3, 4, 0, range(-5,0)
>>> x, math.y
(3, 4)
>>> s
[-5, -4, -3, -2, -1, 5, 6, 7, 8, 9, 10, 0]
```

```
var = expr
obj.attr = expr
obj[idx] = expr
obj[slice] = iterable
```

- 序列解包还可以嵌套

```
>>> a, [b, (c, d)] = 1, ['hello', ('Steve', 'Lee')]
>>> a, b, c, d
(1, 'hello', 'Steve', 'Lee')
```

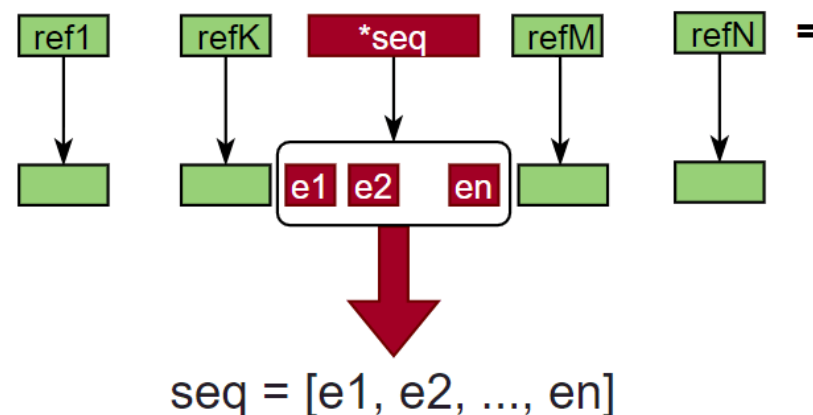


扩展序列解包

扩展序列解包用于赋值语句LHS=RHS，相比原来的序列解包

- 引入**带星号的对象引用 (*seq)**
- LHS中**最多允许出现一次带星号的对象引用**，该引用前后的变量——对应赋予右边的可迭代对象的元素之后，**剩余的元素都转变为list**然后赋值给该引用seq

```
>>> a, *b, c = range(1, 7)
>>> a, b, c
(1, [2, 3, 4, 5], 6)
```



```
data = [ 'Tom', 'math', '16010045', '13905750074', (1999, 12, 21) ]
name, major, _, phone, birth = data
```

```
record = ('Dave', 'dave@example.com', '773-555-1212', '847-555-1212')
name, email, *phone_numbers = record
print(name, email, phone_numbers)
```

iterable_unpacking.py

```
# 不采用扩展序列解包的实现如下所示，要求record支持下标和切片等
name, email, phone_numbers = record[0], record[1], record[2:]
```

序列解包

- 序列解包不仅仅用于**赋值语句(包括for循环中隐含的赋值: for x, y in iterable:)**

- 序列解包还可用于**函数调用**, 称为iterable argument unpacking:

- *seq** 将可迭代对象seq的各个元素拆分后作为下一组**位置实参**

```
s = list(range(10))  
print(*s, sep=' + ', end=' = ')  
print(sum(s)) # 0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 = 45
```

- 序列解包可用于序列**字面量定义**中 (python 3.5引入)

*seq 表示将可迭代对象seq的各个元素拆分为多个元素

```
>>> *range(4), 4          # 元组字面量定义  
(0, 1, 2, 3, 4)  
>>> [*range(4), 4]       # 列表字面量定义  
[0, 1, 2, 3, 4]  
  
>>> {*range(4), 4}       # 集合字面量定义  
{0, 1, 2, 3, 4}
```

func(x, y, *seq, ...)

e1 e2 eN



func(x, y, e1, e2, ..., eN, ...)

[a, b, *seq, x, y]

e1 e2 eN



[a, b, e1, e2, ..., eN, x, y]

序列解包总结

使用场景	描述
<code>x, y, z = range(3)</code>	赋值语句 的基本序列解包，支持嵌套序列解包
<code>x, y, z, *seq = range(10)</code>	赋值语句 的扩展序列解包，支持嵌套序列解包。首先匹配其他位置的对象引用，剩余的元素作为seq所指向的新列表中的元素
<code>func(x, y, *seq)</code>	函数调用时 的序列解包，将可迭代对象seq的各个元素拆分后作为位置实参
<code>[*range(4), 4]</code>	元组、列表、集合 字面量定义时 的序列解包，将可迭代对象的各个元素拆分后作为新的对象中的元素
<code>func(x, y, *seq, **map)</code>	函数调用时的 映射解包 ，表示将map对象(如字典)的各个元素拆分后作为关键实参 (var=value)
<code>{'x': 1, **{'y': 2}}</code>	字典字面量定义时的 映射解包 ，将map对象的各个元素(key:value)拆分后作为新的字典对象中的元素

```
print(*range(4))           # 1
k = *range(4)               # 2 SyntaxError, 不是字面量定义
k = *range(4),              # 3 元组字面量定义的序列解包
```

判断素数时，可能的因子：2以及指定范围内的奇数，即(2, *range(3, int(math.sqrt(n))+1, 2)

序列解包使用实例

问题8-5变种：编写函数，接收整数参数t，返回斐波那契数列中大于t的第一个数。

斐波那契数列： 1, 1, 2, 3, 5, 数列中的某一项等于前面两项之和

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$

1	1	2	3		prev	curr		
fib(1)	fib(2)	fib(3)	fib(4)	...	fib(n-2)	fib(n-1)	fib(n)	fib(n+1)
					fib(n-1) fib(n)			
					prev curr			

fib.py

```
def fib(t):
    """返回大于t的最小fibonacci数"""
    prev, curr = 1, 1
    while curr <= t:
        #         print(prev, end=' ')
        prev, curr = curr, prev + curr
    return curr
```

序列解包使用实例

编写函数，求两个正整数的最大公约数和最小公倍数

接收两个正整数(比如24,60)作为参数，返回一个元组，其中第一个元素为最大公约数，第二个元素为最小公倍数

- 最大公约数(最大公因数或最大公因子) greatest division factor: 两个或者多个自然数的共有的因数中最大的一个
- 最小公倍数(Least Common Multiple): 两个或者多个自然数的共有的倍数中除0之外的最小的一个
- 最大公因数和最小公倍数之间的性质: 两个自然数的乘积等于这两个自然数的最大公约数和最小公倍数的乘积
- **辗转相除法** (欧几里德法) 求最大公约数 $\text{gcd}(m, n)$
 - 如果 $m=0$, 则 $\text{gcd}(0, n) = n$
 - $\text{gcd}(m, n) = \text{gcd}(n \% m, m)$

$\text{gcd}(m, n)$		$\Rightarrow$	$\text{gcd}(n \% m, m)$
93	42		$42 \% 93, 93$
42	93		$93 \% 42 = 9$
9	42		$42 \% 9 = 6$
6	9		$9 \% 6 = 3$
3	6		$6 \% 3 = 0$
0	3		

```
def gcd(m, n):  
    p = m * n  
    while m != 0:  
        m, n = n % m, m  
    return n, p // n
```

gcd.py

如果要求多个整数的最大公约数呢? $\text{gcd}(\text{iterable})$
比如: $\text{gcd}((30, 75, 255)) = 15$

序列解包使用实例

问题7-7 编写程序，判断今天是今年的第几天

1. 首先得到今天对应的年份、月份和当月第几天
 2. 今年的第几天=前面的月份天数之和+当月第几天
- 2月的天数随闰年而不同, 怎么判断闰年?
 - 能被400整除
 - 或者能被4整除, 但不能被100整除

0				11	
31	28	31	...	30	31
month - 1					

```
def is_leap_year(year):  
    return year % 400 == 0 or year % 4 == 0 and year % 100 != 0  
  
def day_of_year(year, month, day):  
    DAYS_IN_MONTH = 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31  
    if month == 1:  
        return day  
    days = sum(DAYS_IN_MONTH[: month - 1]) + day  
    if month > 2 and is_leap_year(year):  
        days += 1  
    return days
```

```
import time  
date = time.localtime()  
year, month, day, *_ = date  
days = day_of_year(year, month, day)  
print(f"{year:4d}-{month:02d}-{day:02d} 是今年的第{days}天")
```

day_of_year.py

```
>>> import time  
>>> time.localtime() # 9个元素的元组  
time.struct_time(tm_year=2025,  
tm_mon=11, tm_mday=11, tm_hour=14,  
tm_min=9, tm_sec=32, tm_wday=1,  
tm_yday=315, tm_isdst=0)
```

主要内容

- 有序对象（字符串、range对象、列表和元组）的通用方法（第4章, 5.1, 5.2）
- 可变有序对象（列表）（第4章）
- 有序对象的切片（第6章）
- 序列解包（2.4）
- **用于序列的常用内置函数（4.8）：zip和enumerate**
- 函数式编程：
 - 列表推导式（6.5-6.7）
 - 生成器表达式
 - 高阶函数（12.6, 12.7）

用于序列对象的常用内置函数

- `len(seq)`: 返回容器对象seq中的元素个数, 适用于列表、range、元组、字典、集合、字符串等
- `max`和`min`: 两种语法
 - `max(arg1, arg2, *args, key=func)`、`min(arg1, arg2, *args, key=func)`: 传递的参数中的最大或最小值
 - `max(iterable, default=obj, key=func)`、`min(iterable, default=obj, key=func)`: 返回迭代对象中的最大或最小**元素**, 如果为空, 返回obj
- `sum(iterable, start=0)`: 对**数值型可迭代对象**的元素进行求和运算, 最后加上start (缺省为0)。**元素为非数值型时抛出异常TypeError**

```
>>> s = [1, 4, -5, -7, 6]
>>> len(s)
5
>>> max(1, 4, -5, -7, 6)
6
>>> min(1, 4, -5, -7, 6)
-7

>>> max(s)
6
>>> min(s)
-7
>>> max([1,2], [2,4])
[2, 4]

>>> sum(s)
-1
```

用于序列对象的常用内置函数:zip

zip(iter1,iter2,...):返回一个**zip对象**, 该对象是一个迭代器(iterator)对象

- 传递的参数为可迭代对象(列表、字符串、元组、range等, 甚至包括字典)
- 每次调用next(zip_object)会返回一个元组, 该元组的元素为各个参数中对应位置的对象, 第一次为所有参数的第一个元素, 第二次为所有参数的第二个元素...
- 如果这些可迭代对象的长度不一致, 则该迭代器的元素个数为**这些长度的最小值**

```
>>> a = [1, 2, 3]
>>> b = [4, 5, 6]
>>> c = zip(a, b)
>>> c
<zip object at 0x0000000003728908>
>>> list(c)
[(1, 4), (2, 5), (3, 6)]
>>> list(c)      # why???
[]
```

zip函数的参数为可迭代对象

itera	iterb	iterc
itera_1	iterb_1	iterc_1
itera_2	iterb_2	iterc_2
itera_3	iterb_3	iterc_3
itera_4		iterc_4
itera_5		

返回可迭代的zip对象

zip(itera, iterb, iterc)
itera_1, iterb_1, iterc_1
itera_2, iterb_2, iterc_2
itera_3, iterb_3, iterc_3

zip对象可以进一步用list/tuple等函数转换为列表/元组对象

zip(iter1,iter2,...)返回的**zip对象**, 是一个**迭代器(iterator)对象**

- 调用list(...)后将zip对象中的元素取完了
- 再次调用时还是试图取zip对象的下一个元素

zip实例：同时对多个序列进行迭代

```
for item in zip(xpts, ypts):  
    x, y = item
```



```
for x, y in zip(xpts, ypts):  
    pass
```

```
xpts = (1, 5, 4, 2, 10, 7)  
ypts = [101, 78, 37, 15, 62, 99, 100]  
for x, y in zip(xpts, ypts):  
    print(x, y, sep='\t')
```

1	101
5	78
4	37
2	15
10	62
7	99

zip_enumerate.py

- 每次取zip对象的下一个元素，即来自于xpts和ypts中对应位置的元素组成的元组，假设为 xpts_i, ypts_i
- 序列展开: x, y = xpts_i, ypts_i

拓展：itertools模块包含许多迭代器，zip_longest(iter1, iter2, ..., fillvalue=None) 返回zip_longest对象

- 与zip类似，只是迭代器的元素个数为所有可迭代对象参数的元素个数的最大值
- 如果某个可迭代对象参数没有更多的元素了，相当于下一个元素为fillvalue，缺省为None

```
import itertools  
s1 = 'abcde'  
s2 = list(range(3))  
for x, y in itertools.zip_longest(s1, s2, fillvalue=0):  
    print(x, y, sep='\t')
```

a	0
b	1
c	2
d	0
e	0

用于序列操作的常用内置函数:enumerate

- `enumerate(iterable[, start])`: 返回`enumerate`(枚举)对象, 是一个迭代器对象, 其每个元素为包含下标和对应可迭代对象的元素的元组。第一个元素的下标从`start`开始, 缺省为0

```
d_list = [5, 6, 7]
for item in enumerate(d_list):
    print(item)

for idx, value in enumerate(d_list, 1):
    print(idx, value)

data = zip([1, 3, 5], [2, 4, 6])
# Error!
# for n, x, y in enumerate(data):
for n, (x, y) in enumerate(data):
    print(n, ': ', x, y)
```

zip_enumerate.py

```
(0, 5)
(1, 6)
(2, 7)
```

```
1 5
2 6
3 7
```

```
0:1 2
1:3 4
2:5 6
```

```
for i in range(len(d_list)):
    print(i, d_list[i])
```

可迭代对象iter1

0	e1
1	e2
2	e3
3	e4
4	e5

`enumerate(iter1)`

0, e1
1, e2
2, e3
3, e4
5, e5

`enumerate(iter1, start)` 下标从`start`开始

`idx, val = (index, value)` #序列解包

```
for n, (x, y) in enumerate(data): pass
# enumerate(data)的元素为: index, (iter1_i, iter2_i)
# 采用嵌套序列解包:
# n, (x, y) = index, (iter1_i, iter2_i)
```

enumerate使用示例

编写函数，参数为多个整数组成的有序对象，查找其中的最小值及其下标(可能有多个)。

返回一个元组，其中第一个元素为最小值，其余元素为最小值在s中的下标（最小值出现的位置）

min_indices.py

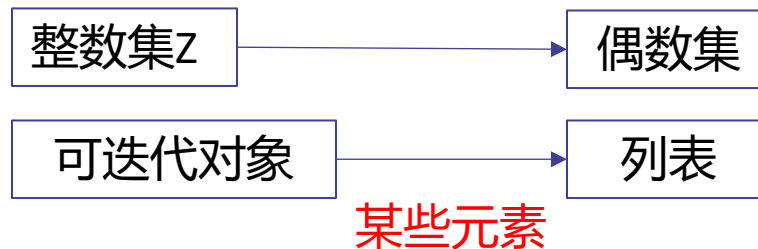
```
def find_min_indices(s):  
    min_value = min(s)  
    result = [min_value]  
    for index, value in enumerate(s):  
        if value == min_value:  
            result.append(index)  
    return tuple(result)
```

主要内容

- 有序对象（字符串、range对象、列表和元组）的通用方法（第4章, 5.1, 5.2）
- 可变有序对象（列表）（第4章）
- 有序对象的切片（第6章）
- 序列解包（2.4）
- 用于序列的常用内置函数（4.8）：zip和enumerate
- 函数式编程：
 - **列表推导式（6.5-6.7）**
 - 生成器表达式
 - 高阶函数（12.6, 12.7）

列表推导式 (list comprehension)

- 集合论: $\{x \mid p(x)\}$
 - 偶数 $\{x \mid x = 2k, k \in \mathbb{Z}\}$
 - 平方数 $\{x \mid x = k^2, k \in \mathbb{Z}\}$ 或者 $\{k^2 \mid k \in \mathbb{Z}\}$
- 列表推导式/解析式 (list comprehension) 是利用其他可迭代对象(比如列表)中的各个元素或者某些元素**创建新列表**的一种方法, 非常简洁, 速度快(底层用C语言实现), 代码具有强可读性



`[expr for value in iterable]`

`[2*k for k in range(100)]`

`[expr for value in iterable if condition]`

`[2*k for k in range(100) if k % 10 == 0]`

基本要素:

- **可迭代对象**中的元素, 通过**表达式进行进一步运算**, 对应新列表中的元素
- if子句**过滤掉**可迭代对象中的一些元素
- **列表推导式中引入的变量相当于本地变量, 仅在列表推导式中有意义**

`s = [x * x for x in range(10)]` `s = [x * x for x in range(10) if x % 2]`

`# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]`

`# [1, 9, 25, 49, 81]`

```
s = []
for x in range(10):
    s.append(x * x)
```

```
s = []
for x in range(10):
    if x % 2:
        s.append(x * x)
```

列表推导式：复杂表达式

列表推导式中的表达式可是函数调用、元组列表等字面量定义、复杂表达式, 还可以是列表解析式

- **函数调用**

```
>>> v = [-2, 5.3, 7, 8.6, -7.4, -9, 10.2]
>>> [abs(x) for x in v if isinstance(x, int)]
[2, 7, 9]
```

- **元组定义, 每个元素为(数, 数的平方)**

```
>>> [(abs(x), x**2) for x in v if isinstance(x, int)]
[(2, 4), (7, 49), (9, 81)]
```

- **if else三元表达式**

```
>>> [x if x >= 0 else -x for x in v if isinstance(x, int)]
[2, 7, 9]
```

- **列表解析式, 构造一个二维矩阵 (列表的列表)**

```
>>> [[i * j for j in range(1, 4)] for i in range(1, 4)]
[[1, 2, 3], [2, 4, 6], [3, 6, 9]]
```

```
[[1, 2, 3],
 [2, 4, 6],
 [3, 6, 9]]
```

列表推导式示例

问题6-1采用列表推导式实现 → 问题6-5： 筛选字符串。筛选出以ly结尾的单词，并统计其数量

```
def count_ly_words(words):  
    ly_words = []  
    for word in words:  
        if word[-2:] == "ly":  
            ly_words.append(word)  
  
    # print("共有%d个单词以ly结尾:" % len(ly_words))  
    print(f"共有{len(ly_words)}个单词以ly结尾")  
    print(", ".join(ly_words))
```

```
def count_ly_words2(words):  
    ly_words = [word for word in words if word[-2:] == "ly"]  
    print(f"共有{len(ly_words)}个单词以ly结尾")  
    print(", ".join(ly_words))
```

count_ly_words.py

列表推导式: 多个for/if子句

语法: [expr for value in iterable] 或 [expr for value in iterable if condition]

第一个for(加上可选的if)之后可以跟0或多个for(加上可选的if)

多个for子句语法:

```
[expr for value1 in iterable1 if condition1
     for value2 in iterable2 if condition2
     ...
     for valueN in iterableN if conditionN]
```

产生包含了所有两位偶数的列表

```
s1 = list(range(10, 100, 2)) # 最简单的实现
s2 = [10 * x + y for x in range(1, 10)
      for y in range(0, 10, 2)]
s3 = [10 * x + y for x in range(0, 10) if x != 0
      for y in range(10) if y % 2 == 0]
```

- 从一个迭代对象中取value1
- 在此基础上从第二个迭代对象中取value2
- ...
- 从最后一个迭代对象中取valueN
- 这些value进行运算作为新列表中的元素
- expr: 可以使用所有for变量
- 后面的for子句和if子句可以使用前面的for变量

```
s4 = [ ] # s3对应的循环实现
for x in range(0, 10):
    if x != 0:
        for y in range(10):
            if y % 2 == 0:
                s4.append(10 * x + y)
```

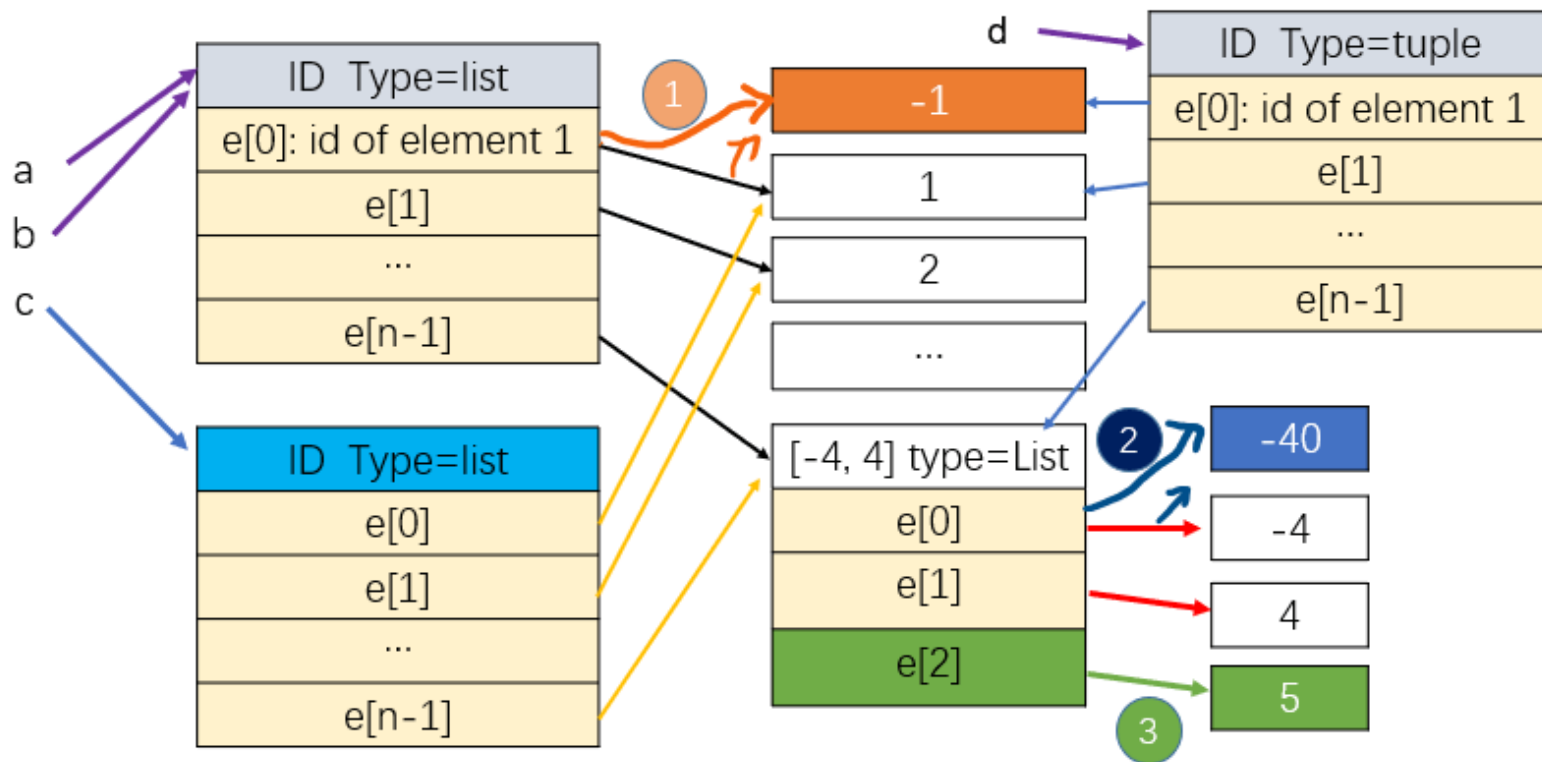
浅拷贝(shallow copy)

- 调用构造函数基于可迭代对象构造一个新的序列对象时，新的序列对象的元素与原可迭代对象中的对应元素指向同一个对象，即构造时采用**浅拷贝(shallow copy)**
- python对于容器对象的许多操作（字面量以及加法、乘法和切片等）缺省采用浅拷贝
- copy模块的**deepcopy**方法进行**深拷贝**，返回同种类型的对象，确保新对象与原对象隔离，即改变某个对象不会影响另一个对象

```
e = (1, 2, 3, [-4, 4])
a = [1, 2, 3, [-4, 4]]
b = a
c = list(a)
d = tuple(a)

a[0] = -1           # 1 不会影响c和d
c[3][0] = -40       # 2 影响a/b和d
c[3].append(5)      # 3 影响a/b和d
```

```
>>> import copy
>>> s = [0, 1, [3, 4, 5]]
>>> s2 = copy.deepcopy(s)
>>> s2
[0, 1, [3, 4, 5]]
>>> s2[-1] is s[-1]
False
```



浅拷贝操作

- python对于容器对象的许多操作（字面量以及加法、乘法和切片等）缺省采用浅拷贝

```
>>> t = (1, 2, [3])
>>> t1 = t * 2
>>> t1
(1, 2, [3], 1, 2, [3])
>>> t[-1][0] = 0
>>> t
(1, 2, [0])
>>> t1
(1, 2, [0], 1, 2, [0])
```

```
>>> row = [0] * 3
>>> m1 = [row, row, row] # row是可变对象
>>> m2 = [row] * 3      # [[0] * 3] * 3
>>> m3 = list(m1)
>>> m4 = [list(row), list(row), list(row)]
```

- m1、m2和m3的3个slot都指向同一个列表对象row
- m4的每个slot都指向不同的列表对象

```
>>> m1
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
>>> m2
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
>>> m3
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
>>> m4
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]

>>> row[0] = -1
>>> m1
[[-1, 0, 0], [-1, 0, 0], [-1, 0, 0]]
>>> m2
[[-1, 0, 0], [-1, 0, 0], [-1, 0, 0]]
>>> m3
[[-1, 0, 0], [-1, 0, 0], [-1, 0, 0]]
>>> m4
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
```

二维矩阵 (Matrix) (8.5)

- 可以通过列表的列表来表示二维矩阵(matrix): `matrix = [row1, row2, ... row_n]`
 - matrix的元素为每一行对应的列表, 该列表中的元素为该行的各列
 - `len(matrix)`为矩阵的行数, 而`len(matrix[0])`为矩阵的列数
 - 通过多级下标(行下标和列下标)访问矩阵中的元素, `matrix[i][j]`表示第i+1行和第j+1列的元素

matrix = [		[0]	[1]	[2]	[3]	[4]
[1, 2, 3, 4, 5],	[0]	1	2	3	4	5
[6, 7, 0, 0, 0],	[1]	6	7	0	0	0
[0, 1, 0, 0, 0],	[2]	0	1	0	0	0
[1, 0, 0, 0, 8],	[3]	1	0	0	0	8
[0, 0, 9, 0, 3],	[4]	0	0	9	0	3
]						

`matrix[0]` → [1, 2, 3, 4, 5]

`matrix[1]` → [6, 7, 0, 0, 0]

`matrix[2]` → [0, 1, 0, 0, 0]

`matrix[3]` → [1, 0, 0, 0, 8]

`matrix[4]` → [0, 0, 9, 0, 3]

`matrix[0][0]` → 1

`matrix[4][4]` → 3

- 初始化矩阵: 全0矩阵、随机矩阵、单位矩阵、用户的输入
- 输出(显示) 矩阵
- 访问矩阵的行和列, 矩阵的平铺(flatten)
- 转置矩阵

矩阵初始化 问题8-26: 初始化矩阵 + 输出矩阵

- 产生指定维度的随机矩阵

matrix_usage.py

```
import random
def generate_random_matrix(rows=3, cols=3):
    matrix = [] # Create an empty list
    for i in range(rows):
        row = []
        for j in range(cols):
            row.append(random.randint(0, 100))
        matrix.append(row)
    return matrix
```

```
matrix = [[random.randint(0, 100) for i in range(cols)] for i in range(rows)]
```

- 产生指定维度的矩阵, 提醒用户输入相应的值

```
def generate_matrix_from_input(rows, cols):
    return [
        [float(input(f"matrix[{i},{j}] = ")) for j in range(cols)] for i in range(rows)
    ]
```

```
>>> generate_matrix_from_input(2, 3)
matrix[0,0] = 45.6
matrix[0,1] = 78
...
```

矩阵初始化 问题8-26: 初始化矩阵 + 输出矩阵

- 产生指定维度的随机矩阵

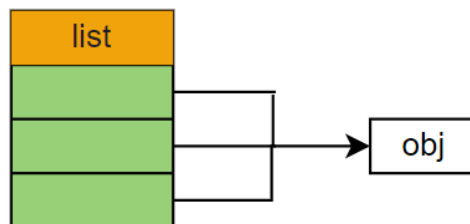
```
matrix = [[random.randint(0, 100) for i in range(cols)] for i in range(rows)]
```

- 产生全0矩阵

```
def generate_zeros_matrix(rows=3, cols=3):  
    matrix = [[0] * cols for i in range(rows)]  
    # 错误 matrix = [[0] * cols] * rows  
    return matrix
```

`[0] * cols` 等价于
`[0 for i in range(cols)]`

```
# 错误的代码  
matrix = [[0] * 4] * 3  
  
row = [0] * 4  
matrix = [row, row, row]
```



二维矩阵的访问

- 显示矩阵

```
def print_matrix(matrix):  
    for row in matrix:  
        for value in row:  
            print(f'{value:5}', end="")  
        print() # Print a new line
```

```
def print_matrix2(matrix): # 通过下标访问  
    for i in range(len(matrix)):  
        for j in range(len(matrix[i])):  
            print(f'{matrix[i][j]:5}', end="")  
        print() # Print a new line
```

- 获得矩阵的第i行: `matrix[i - 1]`
- 获得矩阵的第j列:

```
def get_column(matrix, column=0):  
    vec = []  
    for row in matrix:  
        vec.append(row[column])  
    return vec
```

```
def get_column2(matrix, column=0):  
    return [row[column] for row in matrix]
```

	[0]	[1]	[2]	[3]	[4]
[0]	1	2	3	4	5
[1]	6	7	0	0	0
[2]	0	1	0	0	0
[3]	1	0	0	0	8
[4]	0	0	9	0	3

列表推导式: 多个for/if子句

6.5.2示例: 实现二维矩阵(嵌套列表)的平铺, 即新列表由最内层的列表元素组成

```
def flatten(matrix):  
    return [value for row in matrix for value in row]
```

```
matrix = [[4 * i + j for j in range(1, 5)] for i in range(3)]  
print(matrix)  
# [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]  
print(flatten(matrix))  
# [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
```

```
def print_matrix(matrix):  
    for row in matrix:  
        for value in row:  
            print(f'{value:5}', end="")  
        print() # Print a new line
```

二维矩阵的转置

6.6 矩阵的转置：把 $m \times n$ 阶矩阵(即 m 行 n 列)的行和列互换，形成 $n \times m$ 阶的转置矩阵。即原矩阵的行变为转置矩阵的列，原矩阵的列变为转置矩阵的行。

- 采用zip函数实现

`zip(*matrix)` 函数调用的序列解包，可迭代对象`matrix`的元素展开为位置实参，相当于 `zip(matrix[0], matrix[1], ..., matrix[len(matrix)-1])`

```
matrix = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]
```

```
matrix_transpose = list(zip(*matrix))
```

```
print(matrix_transpose)
```

```
matrix_transpose2 = [list(item) for item in matrix_transpose]
```

```
matrix_transpose3 = [list(item) for item in zip(*matrix)]
```

```
return matrix_transpose3
```

```
[1, 2, 3, 4],
```

```
[5, 6, 7, 8],
```

```
[9, 10, 11, 12]
```



```
[1, 5, 9 ],
```

```
[2, 6, 10],
```

```
[3, 7, 11],
```

```
[4, 8, 12]
```

```
[(1, 5, 9), (2, 6, 10), (3, 7, 11), (4, 8, 12)]
```

```
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

- 使用列表推导式的嵌套实现矩阵转置

```
>>> matrix = [ [1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]
```

```
>>> [[row[i] for row in matrix] for i in range(len(matrix[0]))]
```

```
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

表达式为一个列表推导式：获得第 i 列
`[row[i] for row in matrix]`

0	1	2	3
<code>[row[0] for row in matrix]</code>	<code>[row[1] for row in matrix]</code>	<code>[row[2] for row in matrix]</code>	<code>[row[3] for row in matrix]</code>

列表推导式：素数

判断p是否为素数

- 一个数是素数，除了1和自身外，没有其他因子
- 首先得到p除以可能的因子（ $[2, \sqrt{n}]$ ）的余数组成的列表
- 如果其中有0，说明不是素数。如果都不为0，则为素数

```
prime = 0 not in [p % d for d in range(2, int(math.sqrt(p) + 1))]
```

使用列表推导式生成100以内的所有素数：

```
>>> import math
>>> [p for p in range(2, 100) if 0 not in
      [p % d for d in range(2, int(math.sqrt(p) + 1))]]
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79,
83, 89, 97]
```

生成器表达式

- **生成器表达式(Generator Expression, 教材P132表示为生成器推导式)**的语法与列表推导式基本相同, 唯一的区别就是生成器表达式使用**圆括号**, 列表推导式使用方括号
- 生成器表达式运算的结果并不是一个元组, 而是一个生成器对象
- 生成器对象是一个**迭代器对象**, 当然也是一个可迭代对象, 可用for循环访问, 也可转换为列表等
- 如果生成器表达式是**函数调用的唯一参数**时, 生成器表达式的圆括号也可以省略

```
>>> [x * x for x in range(3)]
[0, 1, 4]
>>> g = (x * x for x in range(3))
>>> g
<generator object <genexpr> at 0x00FFADB0>
>>> dir(g)
[... '__iter__', ..., '__next__', ...]
>>> next(g)
0
>>> next(g)
1
>>> next(g)
4
```

```
>>> next(g)
Traceback (most recent call last):
  File "#python..", line 1, in <module>
    next(g)
StopIteration
>>> iter(g) is g
True
>>> list(g)
[]
>>> list(x * x for x in range(4))
[0, 1, 4, 9]
>>> enumerate(x * x for x in range(4), 100)
File "<stdin>", line 1
SyntaxError: Generator expression must be parenthesized
```

生成器表达式

勾股数 (pythagorean triplets): 存在 $\{x,y,z\}$, $0 < x < y < z$, 使得 $x^2 + y^2 = z^2$

求前10个勾股数

pyt_firstN.py

```
def pyt_firstN_list(nums):  
    pyt = [(x,y,z) for z in range(1, 100)  
            for y in range(1,z)  
            for x in range(1,y) if x*x + y*y == z*z ]  
    firstN_pyt = pyt[:nums]  
    print(*firstN_pyt)
```

循环次数为 $O(n^3)$, n 从100改成1000?

```
pyt = [(x,y,z) for z in range(1, 1000) for y in range(1,z)  
        for x in range(1,y) if x*x + y*y == z*z ]
```

```
def pyt_firstN_generator(nums):  
    pyt = ((x,y,z) for z in range(1, 10000)  
            for y in range(1, z)  
            for x in range(1, y)  
            if x*x + y*y == z*z )  
    firstN_pyt = [next(pyt) for x in range(nums)]  
    print(*firstN_pyt)
```

主要内容

- 有序对象（字符串、range对象、列表和元组）的通用方法（第4章, 5.1, 5.2）
- 可变有序对象（列表）（第4章）
- 有序对象的切片（第6章）
- 序列解包（2.4）
- 用于序列的常用内置函数（4.8）：zip和enumerate
- 函数式编程：
 - 列表推导式（6.5-6.7）
 - 生成器表达式
 - **高阶函数（12.6, 12.7）**
 - map, filter, reduce和lambda表达式
 - 生成器函数

内置函数map和filter[12.7.1]

在Python语言的发展过程中，是首先有map/filter等内置函数，后来引入列表解析式和生成器表达式

高阶函数：可能接受函数对象作为参数，或者可返回函数对象的参数

map(func, iterable) filter(func, iterable)

- 将一个函数作用到一个可迭代对象上，返回一个**迭代器对象**
- map函数返回的迭代器的元素是对iterable对象的元素调用相应函数func的结果，即每个元素为func(element)
- filter函数返回的迭代器的元素是这样得到的：
 - iterable对象的元素作为参数调用func，如果func(element)返回True，则该元素为迭代器的元素
 - 如果filter函数的第1个参数为None，则iterable对象的元素真值判断为True时作为迭代器的元素

map(func, iterable)	<=====>	(func(item) for item in iterable)
filter(func, iterable)	<=====>	(item for item in iterable if func(item))
filter(None, iterable)	<=====>	(item for item in iterable if item)

```
>>> list(map(str, range(5)))
['0', '1', '2', '3', '4']
>>> s = 1, 2, '', 0, -1, []
>>> list(filter(None, s))
[1, 2, -1]
```

```
>>> def is_even(x):
    return x % 2 == 0

>>> list(filter(lambda x: x % 2 == 0, range(10)))
[0, 2, 4, 6, 8]
>>> list(map(lambda x: 2*x, range(10)))
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

回顾：lambda表达式

lambda表达式用来定义一个仅包含一行代码的匿名函数，运算结果为一个匿名**函数对象**

- 没有函数名字，不需要额外的取名烦恼（可能重复），避免了可能的名字冲突
- 函数体只包含一个表达式，且该表达式的计算结果为函数的返回值
 - 只能是表达式，但注意函数调用也是表达式，因此可以调用print、map、list等。
 - 不允许包含if/for/while等其他复杂的语句
- lambda表达式也是函数的一种，参数定义、作用域规则、调用方式与def语句定义的函数完全等同

```
def __anonymous_function__(arguments):  
    return expr
```



```
lambda arguments: expr
```

```
def is_even(x):  
    return x % 2 == 0
```

```
is_even = lambda x: x % 2 == 0
```

```
>>> list(filter(lambda x: x % 2 == 0, range(10)))  
[0, 2, 4, 6, 8]  
>>> list(map(lambda x: 2*x, range(10)))  
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

- lambda表达式的优先级最低，大部分情况下expr不用加上括号，但注意如果表达式中有**省略了圆括号的元组定义时**，容易出现問題。

```
def f(x=0):  
    return x, x ** 2
```

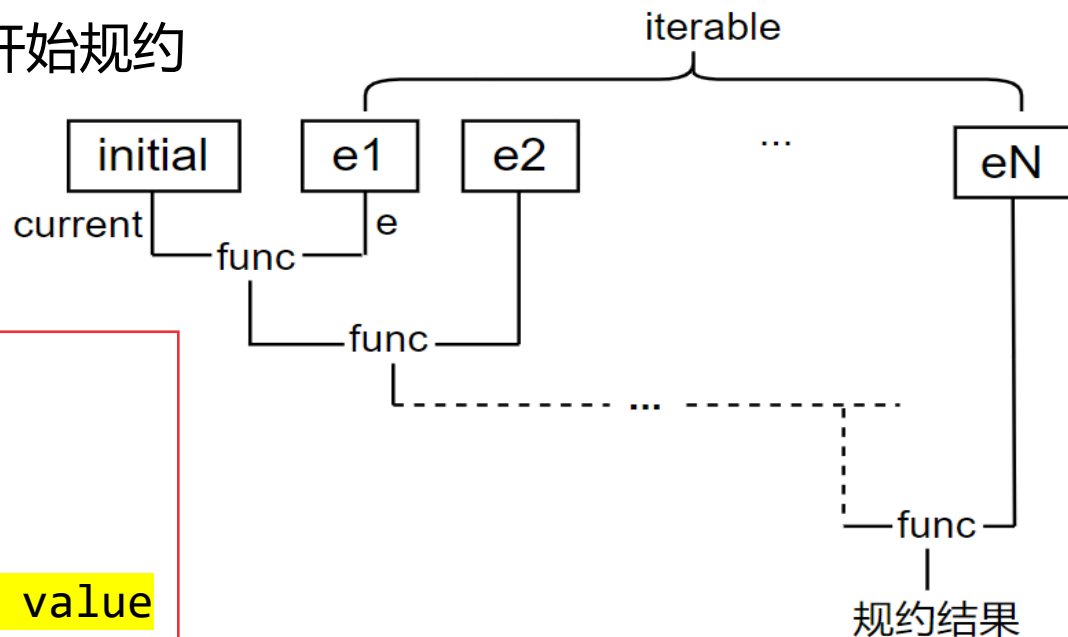
```
# 下面的语句等价  
f = lambda x=0: (x, x ** 2)
```

```
f = lambda x=0: x, x ** 2  
相当于 f = (lambda x=0: x), x ** 2
```

reduce

模块functools的reduce(func, iterable [,initial])

- 将可迭代对象的所有元素进行归约，将它们归并在一起
- func(current, element): 之前规约结果与可迭代对象下一个元素如何规约
- 如果只有2个参数，初始归约结果为第1个元素，从第2个元素开始规约
 - iterable没有元素时，抛出异常`TypeError`
- 如果传递了initial，初始规约结果为initial，从第1个元素开始规约
 - Iterable没有元素时，规约结果为initial



```
>>> from functools import reduce
>>> reduce(lambda x,y:x+y, [])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: reduce() of empty iterable with no initial value
>>> reduce(lambda x,y:x+y, [], 0)
0
```

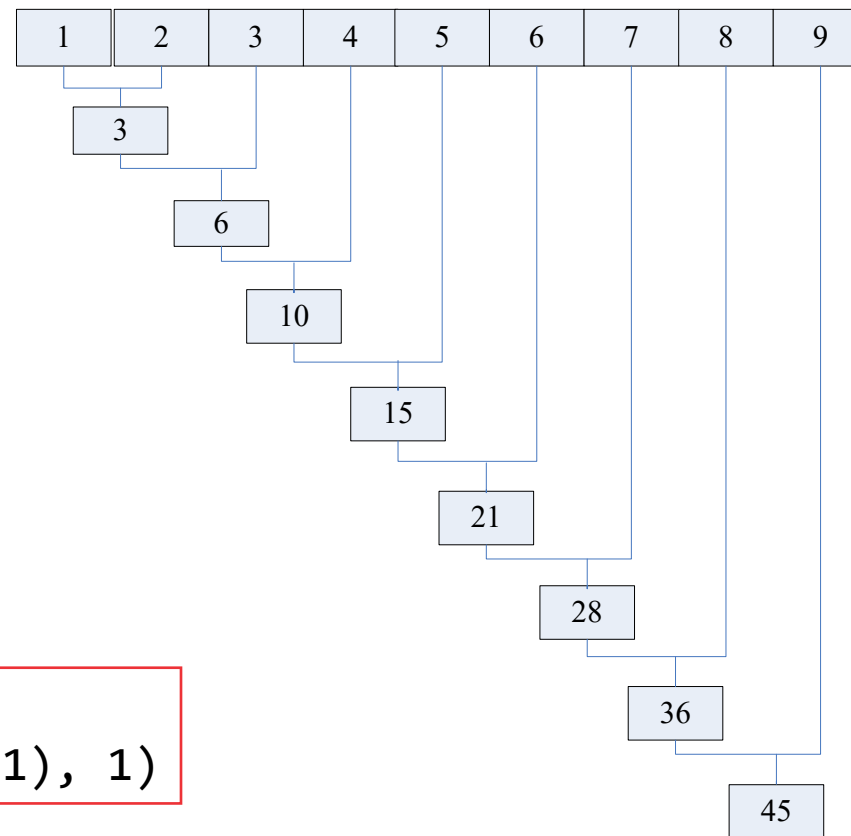
reduce

```
>>> reduce(lambda x,y:x+y, [], 0)  
0
```

`functools.reduce(func, iterable [,initial])`

```
>>> from functools import reduce  
>>> def add(x, y):  
    return x + y  
>>> reduce(add, range(10))  
45  
>>> reduce(lambda x,y:x + y, range(10))  
45  
>>> reduce(lambda x,y:x + y, map(str, range(10)))  
'0123456789'
```

```
def factorial(n):  
    return reduce(lambda x, y: x * y, range(1, n + 1), 1)
```



any和all函数

- any(iterable): iterable中的所有元素中, 如果有任意一个元素真值判断为真, 则返回True。如果iterable为空对象 (长度为0), 则返回False
- all(iterable): iterable中的所有元素中, 如果所有元素的真值判断都为真, 则返回True。如果iterable为空对象, 则返回True

```
def any(iterable):  
    for element in iterable:  
        if element:  
            return True  
    return False
```

```
def all(iterable):  
    for element in iterable:  
        if not element:  
            return False  
    return True
```

```
>>> import random  
>>> import string  
>>> text = ''.join(random.choices(string.printable, k=10))  
# text中是否包含数字?  
>>> any(e.isdigit() for e in text)  
True  
# text是否全是数字  
>>> all(e.isdigit() for e in text)  
False
```

- 生成器表达式: (x ** 2 for x in range(10))
 - 不是一次返回全部数据, 而是一个iterator对象, 通过next内置函数获得下一个元素
- 生成器函数: 生成器表达式的一般化, 给出了如何获取下一个元素的方法
 - **生成器函数**是函数的一种, 普通函数通过return语句返回值, 现在通过yield expression返回值
 - 调用生成器函数时, 传递参数之后暂停运行, 返回一个迭代器对象(生成器), 此时函数体代码并没有执行
 - next(生成器): 等价于调用该生成器的__next__方法, 从上次暂停的地方恢复, 继续执行
 - 执行到yield表达式时, 暂停执行, 将yield后面的表达式求解后的结果作为返回值返回给调用者
 - 执行到return语句, 或者函数体后续没有更多的语句了, 则抛出异常StopIteration
 - **拓展: 恢复执行时, yield表达式的结果为None**

```
def f(iterable):
    for item in iterable:
        yield item ** 2
```

```
g = f(range(3))
print(g)
print(next(g))
print(next(g))
print(next(g))
# print(list(f(range(3))))
```

generator.py

```
<generator object f at 0x00000268C9673780>
```

```
0
```

```
1
```

```
4
```

```
Traceback (most recent call last):
```

```
File "generate_func.py", line 19, in <module>
```

```
    print(next(g))
```

```
    ^^^^^^^
```

```
StopIteration
```

生成器函数

- 原来的累加模式：初始化一个列表，附加元素到列表，最后返回列表
- 生成器函数：不再需要初始化，附加元素改为yield表达式，最后不需要返回列表

```
def find_all_solutions():  
    solutions = []  
    while True:  
        solution = find_next_solution()  
        if done():  
            break  
        solutions.append(solution)  
    return solutions
```

```
def all_solutions_iterator():  
    while True:  
        solution = find_next_solution()  
        if done():  
            break  
        yield solution  
  
for solution in all_solutions_iterator():  
    print(solution)
```

generator_template.py

```
def f0(iterable):  
    s = []  
    for item in iterable:  
        s.append(item ** 2)  
    return s
```

```
def f(iterable):  
    for item in iterable:  
        yield item ** 2  
  
print(list(f(range(10))))
```

generator.py

生成器函数

- yield from iterable: 调用next时返回iterable的下一个元素, 相当于 `for i in iterable: yield i`
- itertools模块中包含了多个迭代器, 包括count/cycle/repeat/chain等

generate_func.py

```
def f1():  
    yield from 'welcome to the python world.'  
  
def f2():  
    for i in 'welcome to the python world.':  
        yield i  
  
def count(start=0, step=1):  
    while True:  
        yield start  
        start += step  
  
# for循环, i从0开始递增到永远, 输出100以内那些10的倍数  
for i in count():  
    if i % 10 == 0:  
        print(i)  
    if i > 100:  
        break
```

生成器函数：斐波那契数列

返回前10个数： 1 1 2 3 5 8 13 21 34 55

- fib(n): 输出小于n的斐波那契数
- fib_iterator(n): 产生斐波那契数列

```
def fib(n):  
    prev, curr = 1, 1  
    while prev < n:  
        print(prev, end=' ')  
        prev, curr = curr, prev + curr  
    print()
```

fib_generator.py

↓

```
def fib_iterator():  
    prev, curr = 1, 1  
    while True:  
        yield prev  
        prev, curr = curr, prev + curr
```

```
def fib_first_n2(n):  
    fibs = fib_iterator()  
    first_n_fibs = [next(fibs) for i in range(n)]  
    print(f'前面{n}个斐波那契数: ', *first_n_fibs)
```

返回1000以内的fibonacci数

```
count = 0  
for i in fib_iterator():  
    if i > 1000:  
        break  
    print(i, end='\t')  
    count += 1  
    if count % 5 == 0 :  
        print()  
if count % 5:  
    print()
```

1	1	2	3	5
8	13	21	34	55
89	144	233	377	610
987				